Final Project – Quanser Aero

Professor: Bernardo Restrepo-Torres

Graduate Assistant: Jared Getz

Project Supervisor: Alexandra Torres

Authors: Anuranan Bharadwaj**,** Kalkamanali Satvaldy, Dobromir Sokolowski

AE 443: Experimental Dynamics & Control Laboratory

Department of Aerospace Engineering

Embry-Riddle Aeronautical University

Daytona Beach, FL, 32114

April 23, 2026

I. Abstract

This report presents the design and evaluation of PID control for the Quanser Aero. The experiment was divided into two parts. In the first part, proportional, derivative, and integral gains were tuned experimentally to study how each control action affected response speed, overshoot, damping, and steady-state error. A final tuned controller of $k_p = 8$, $k_d = 3.1$ and $k_i = 5$ produced the best overall response, with improved tracking of the square-wave input and reduced overshoot. In the second part, the Quanser Aero was approximated as a second-order system using the measured unity-feedback step response. The damping ratio and natural frequency were determined from the experimental data, and these values were used to construct a transfer function and analytically calculate PID gains. The calculated controller was then tested and compared with the manually tuned controller. The results showed that the second-order approximation represented the plant reasonably well but the experimentally tuned controller achieved better overall tracking performance.

II. Procedure

The main goal of section 2 was first to study the open-loop response and then progressively adding proportional, derivative and integral control to improve tracking performance. The changes that were brought with the PID controls were observed and used to satisfy the design requirements for peak time and steady-state values. The response of the Quanser Aero was evaluated using the measured pitch angle while the PID control gains were adjusted.

The procedure began by running the Quanser Aero in an open loop to the baseline step response and response characteristics like peak time, settling time and percentage overshoot were measured. A proportional gain was then added to the loop and varied between 5 to 10 in increments of 2.5 to examine its effects on response speed and oscillation. Next, the derivative gain was added with the proportional gain fixed. Like the proportional gain, the derivative gain was varied between 4 and 8 in increments of 2 to observe and study its effect on damping and overshoot. The input was then changed from a step to a square wave so that the tracking performance could be evaluated. A integral gain was then added to reduce the steady-state error and the gain was varied from 3 to 6 in increments of 1.5 to study what effects it has on the system response. Finally, the proportional, derivative and integral gains were tuned together until the response met the desired performance of overshoot being less than 0.35 radians peak pitch and having a peak time of less than 1.1 seconds

Section 4 procedure focused on designing a PID controller using a second-order system approximation instead of tuning the gains with trial and error. The main goal of this section was to model the Quanser Aero with and equivalent second-order transfer function. System parameters were estimated by using the measured response data and then PID gains were calculated analytically from the desired closed-loop performance requirements.

The procedure for this section began by configuring the Quanser Aero in unity feedback and a unit step input was applied to obtain the experimental response. From the obtained response, peak time, maximum response, steady-state value and percentage overshoot were found. These values were then used to estimate the damping ratio and natural frequency of an equivalent second-order model. By using the equivalent second-order transfer function, the model response was compared with the actual response of Quanser Aero to evaluate how well the approximation represented the plant dynamics. Then by using both the coefficients of the approximated transfer function and the desired critically damped response specifications, the proportional, derivative and integral gains were calculated. These calculated gains were then entered into the controller. The resulting square-wave response of the Quanser Aero was then recorded and analyzed.

III. Results

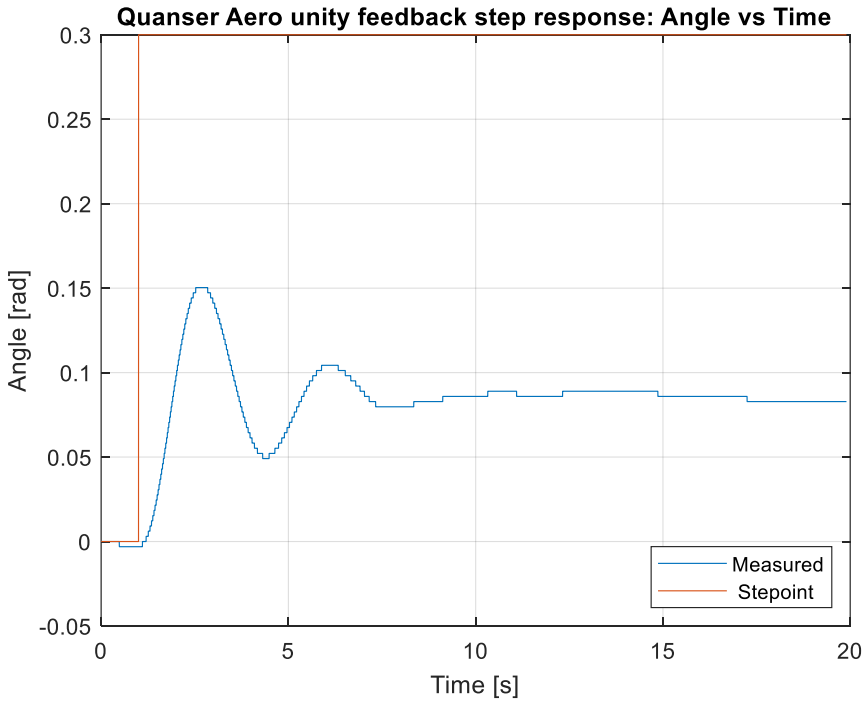


Figure 1: Unity-Feedback Step Response for Section 2.1

Table 1: Response Parameters for the Unity-Feedback Step Response (Section 2.1)

Description	Symbol	Value	Units
Peak Time	t_p	1.694	s
Settling time (5%)	T_s	13.63	s
Percentage Overshoot	PO	68.96	%
Damping Ratio	ζ	0.1178	
Natural Frequency	ω_n	1.865	rad/s

The table above presents a summary of the parameters calculated using the values from Figure 1. As shown in Figure 2, the following points were selected: $t_{peak} = 2.692\ s, t_0 = 0.998\ s, y_{peak} = 0.15033$ and $y_{ss} = 0.0890$. The peak time was calculated using the equation below:

$$t_p = t_{peak} - t_0 = 2.692 - 0.998 = 1.694\ s \quad (1)$$

Percentage Overshoot was calculated using Equation 2 as shown below:

$$PO = \left(\frac{y_{peak} - y_{ss}}{y_{ss}}\right) \times 100 = \left(\frac{0.15033 - 0.0890}{0.0890}\right) \times 100 = 68.96\% \quad (2)$$

To find the damping ratio, Equation 3 below:

$$PO = 100e^{-\frac{\pi\zeta}{\sqrt{1-\zeta^2}}} \tag{3}$$

Was rearranged to Equation 4:

$$\zeta = \frac{-ln(PO)}{\sqrt{\pi^2 + (ln(PO))^2}} = \frac{-ln(0.6896)}{\sqrt{\pi^2 + (ln(0.6896))^2}} = 0.1178 \tag{4}$$

To find the natural frequency, Equation 5 below:

$$t_p = \frac{\pi}{\omega_n\sqrt{1-\zeta^2}} \tag{5}$$

Was rearranged to Equation 6:

$$\omega_n = \frac{\pi}{t_p\sqrt{1-\zeta^2}} = \frac{\pi}{1.694\sqrt{1-0.1178^2}} = 1.865\ rad/s \tag{6}$$

To find the settling time, Equation 7 below:

$$\delta = e^{-\zeta\omega_n T_s} \tag{7}$$

Was rearranged to solve for T_s:

$$T_s = \frac{-ln(\delta)}{\zeta\omega_n} \tag{8}$$

For $\delta = 0.05$:

$$T_s = \frac{-ln(0.05)}{0.1178 * 1.865} = 13.63\ s \tag{9}$$

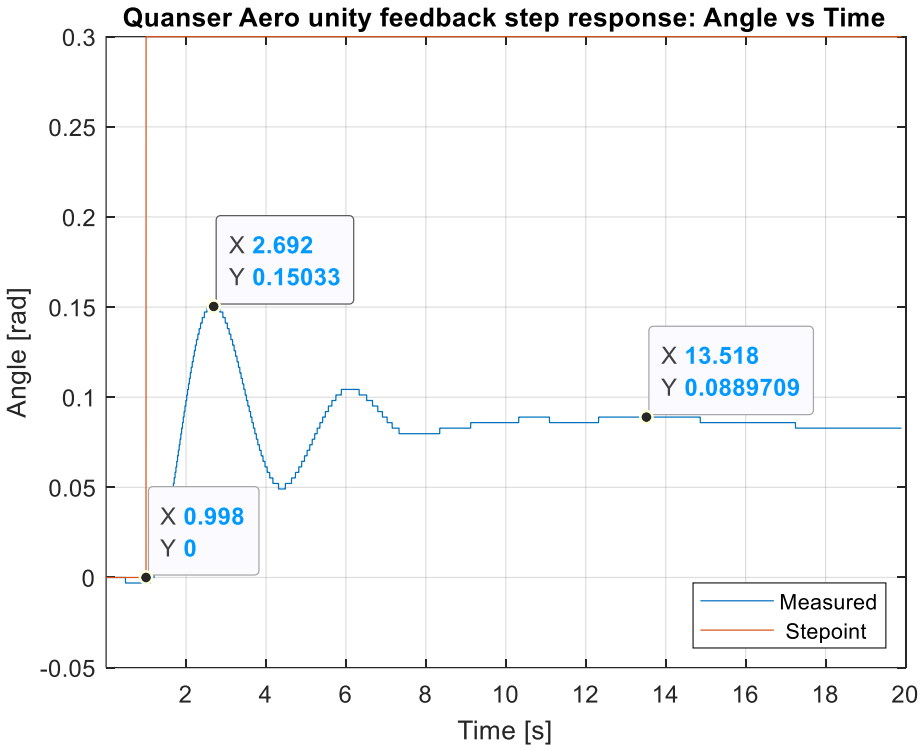


Figure 2: Unity-Feedback Step Response with Selected Data Points for Section 2.1

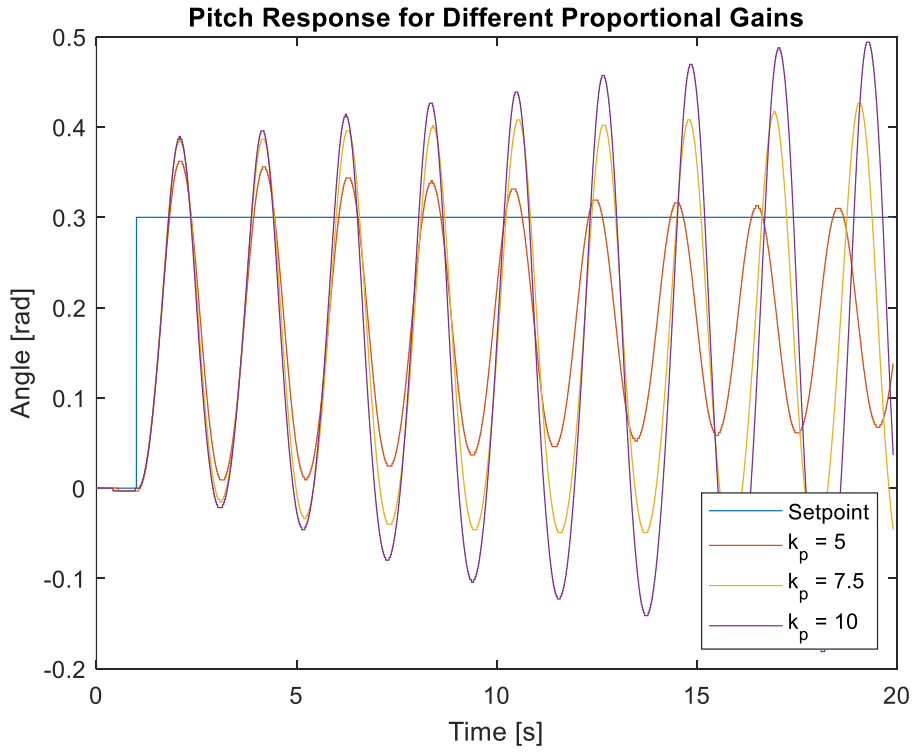


Figure 3: Pitch Response for Different Proportional Gains

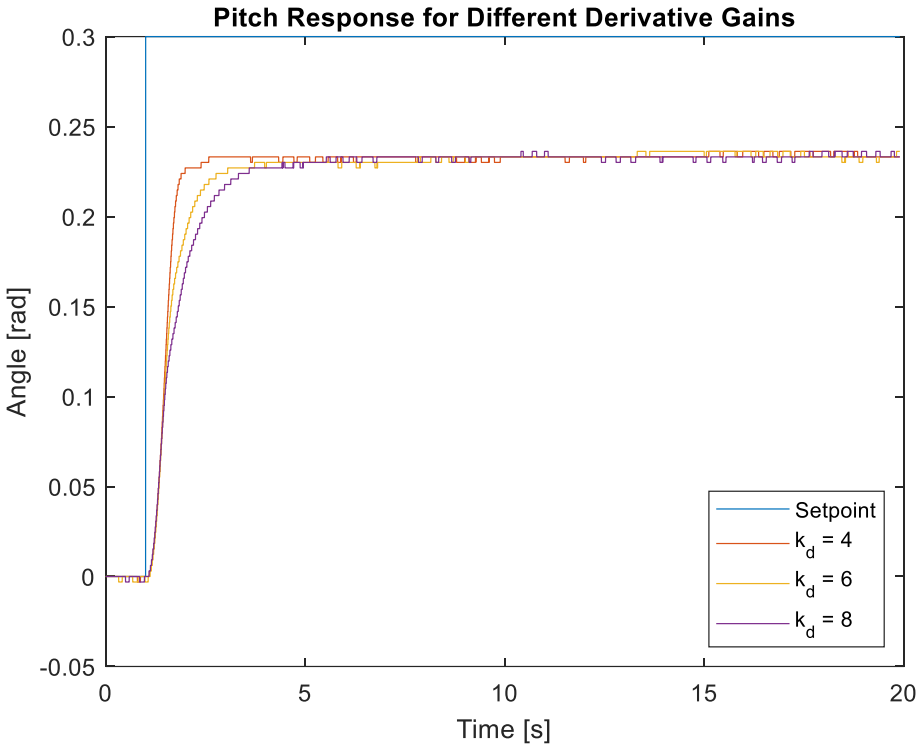


Figure 4: Pitch Response for Different Derivative Gains

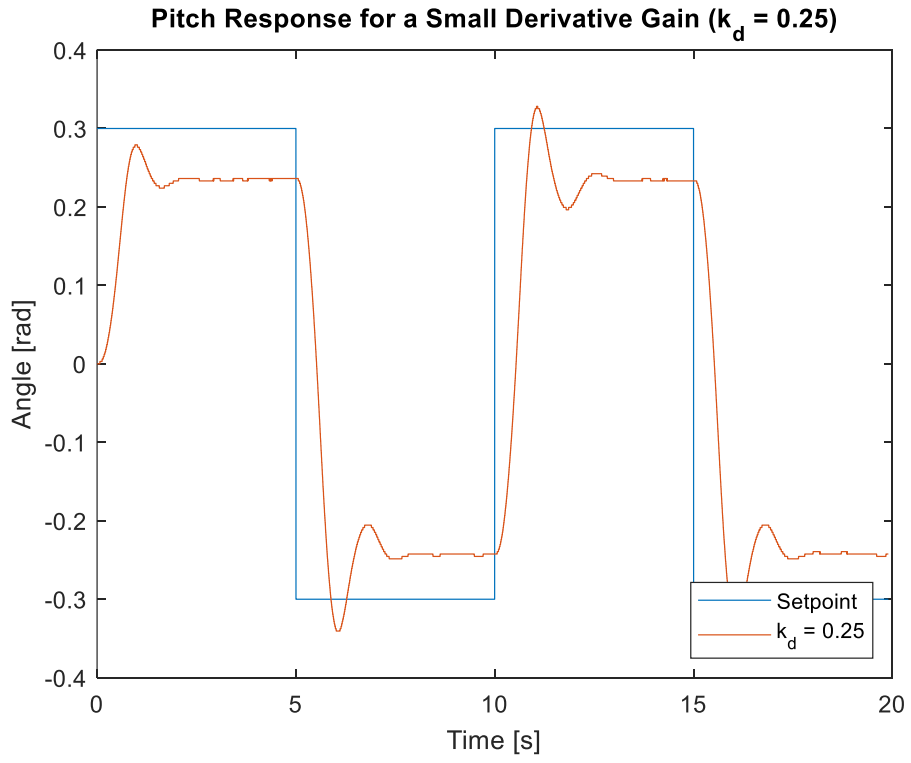


Figure 5: Pitch Response for a Small Derivative Gain ($k_d = 0.25$)

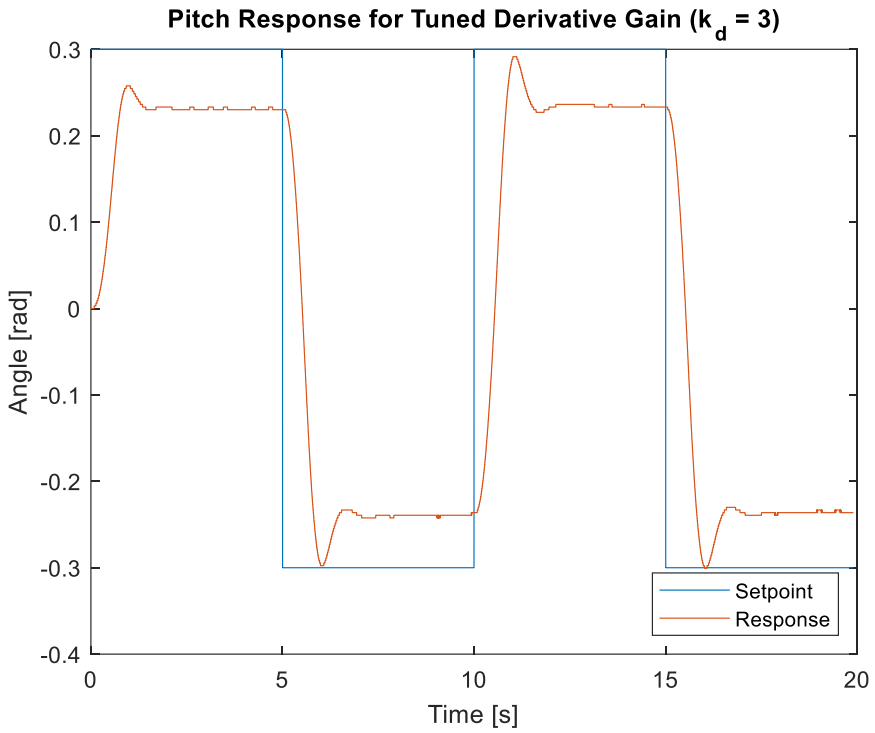


Figure 6: Pitch Response for Tuned Derivative Gain ($k_d = 3$)

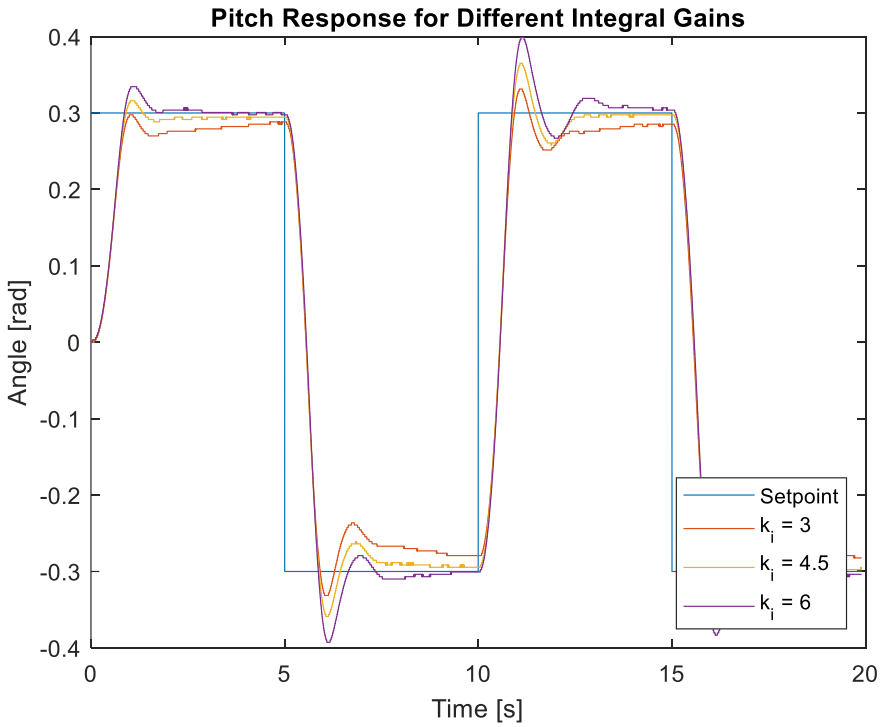


Figure 7: Pitch Response for Different Integral Gains

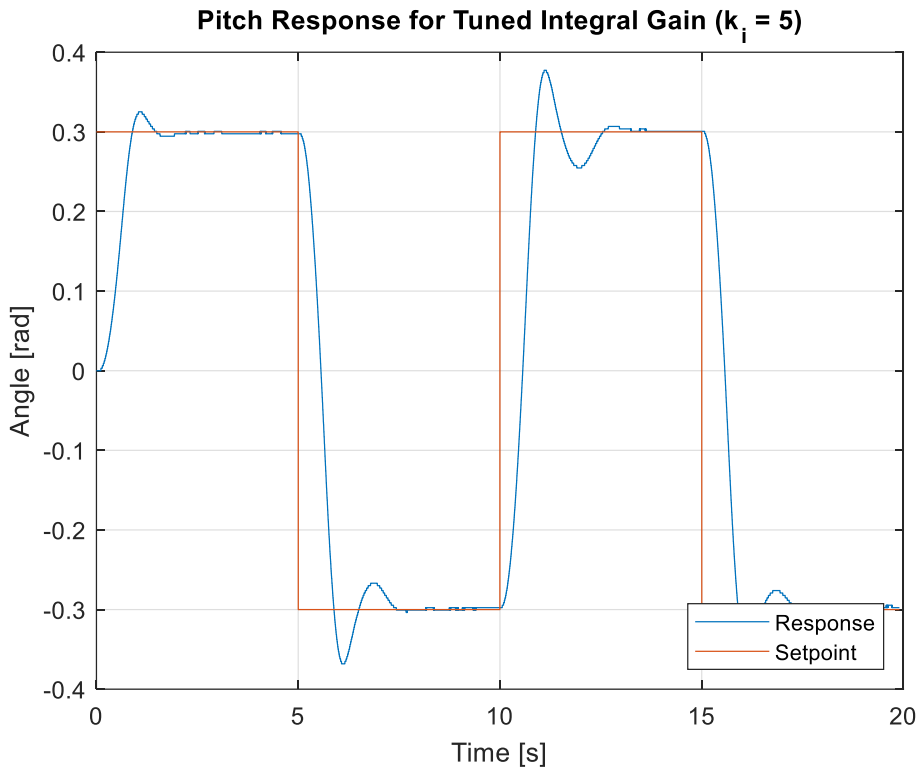


Figure 8: Pitch Response for Tuned Integral Gain ($k_i = 5$)

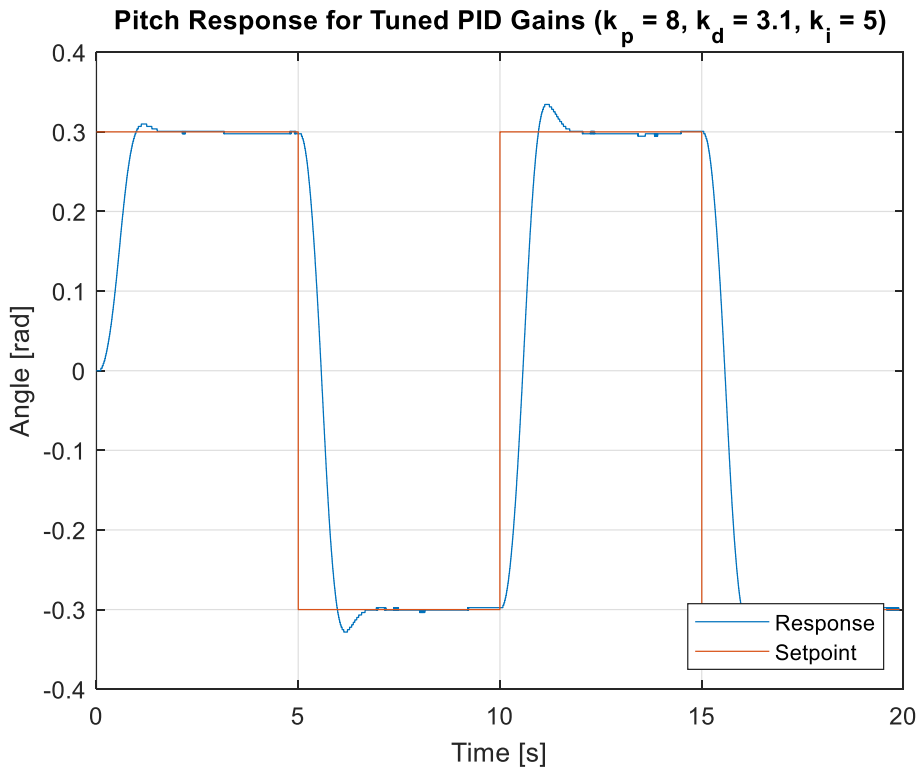


Figure 9: Pitch Response for Tuned PID Gains ($k_p = 8, k_d = 3.1, k_i = 5$)

Table 2 summarizes the tuned derivative gain and peak value from Section 2.3, the tuned integral gain and steady-state value from Section 2.4, and the tuned PID gains with the corresponding overshoot, peak time, and steady-state value from Section 2.5. The values were obtained from Figures 6, 8, and 9, respectively.

Table 2: Summary of Tuned Gain Results for Sections 2.3, 2.4, and 2.5.

Description	Symbol	Value	Units
Section 2.3			
Tuned derivative gain	k_d	3	-
Peak value	y_{peak}	0.2577	rad
Section 2.4			
Tuned integral gain	k_i	5	-
Steady-state value	y_{ss}	0.2976	rad
Section 2.5			
Tuned proportional gain	k_p	8	-
Tuned derivative gain	k_d	3.1	-
Tuned integral gain	k_i	5	-
Peak Time	t_p	1.18	s
Percentage Overshoot	PO	4.12	%
Steady-state value	y_{ss}	0.2976	rad

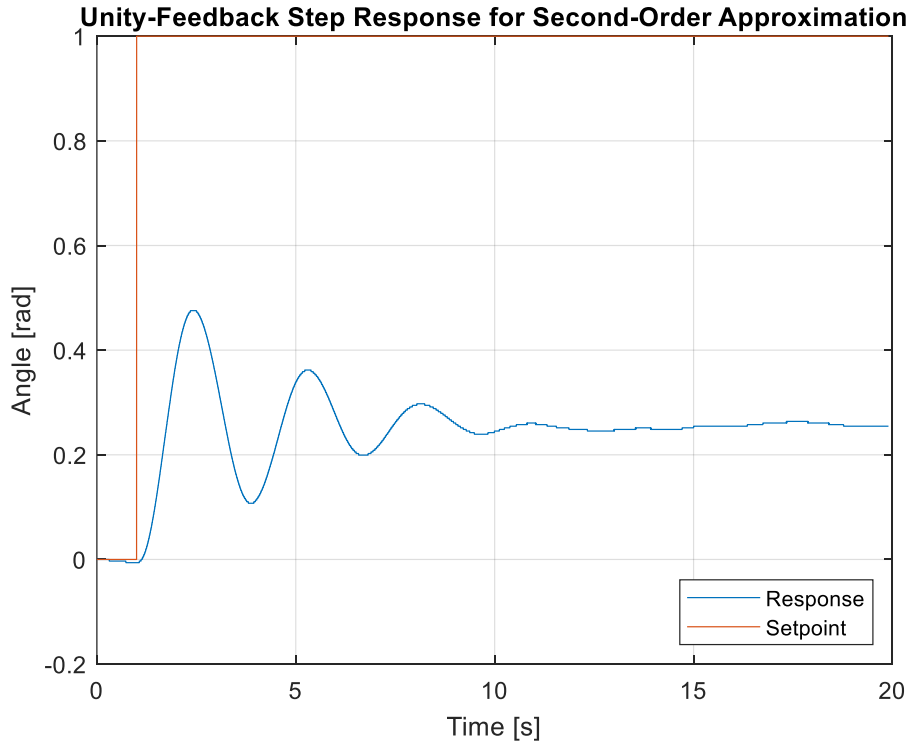


Figure 10: Unity-Feedback Step Response for Second-Order Approximation

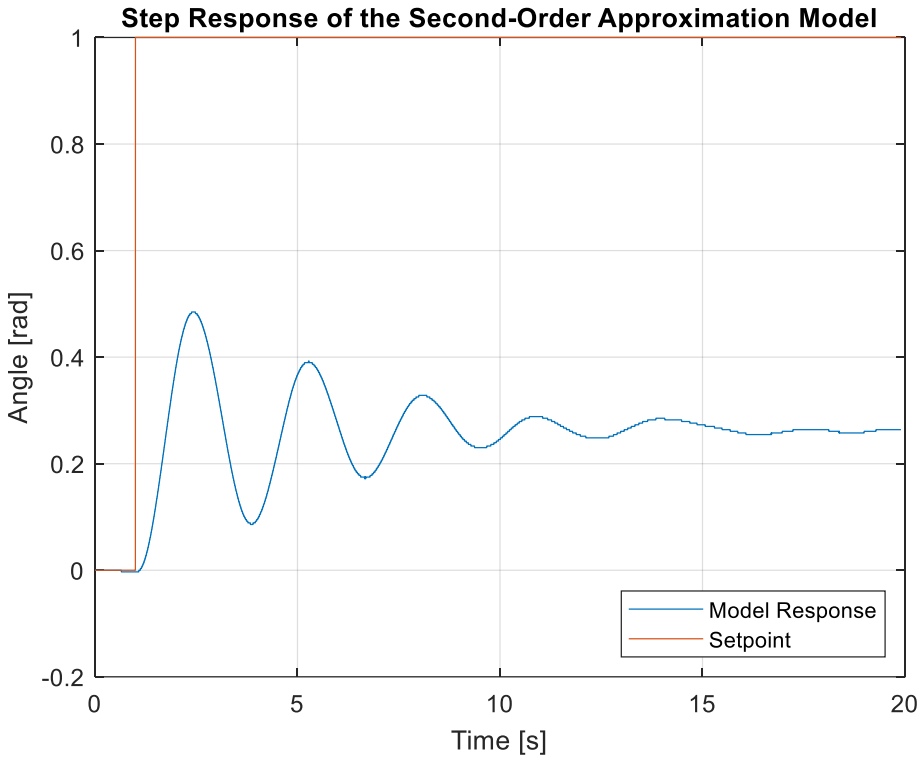


Figure 11: Step Response of the Second-Order Approximation Model

As shown in Figure 12, the following points were selected: $t_{peak} = 2.44\ s$, $t_0 = 0.998\ s$, $y_{peak} = 0.475534\ rad$ and $y_{ss} = 0.254641\ rad$. The peak time was calculated using the Equation 1:

$$t_p = 2.44 - 0.998 = 1.442\ s \tag{10}$$

Percentage Overshoot was calculated using Equation 2 as shown below:

$$PO = \left(\frac{0.475534 - 0.254641}{0.254641}\right) \times 100 = 86.75\% \tag{11}$$

To find the damping ratio, Equation 4 was used as shown below:

$$\zeta = \frac{-ln(0.8675)}{\sqrt{\pi^2 + (ln(0.8675))^2}} = 0.0452 \tag{12}$$

To find the natural frequency, Equation 6 was used as shown below:

$$\omega_n = \frac{\pi}{1.442\sqrt{1 - 0.0452^2}} = 2.18\ rad/s \tag{13}$$

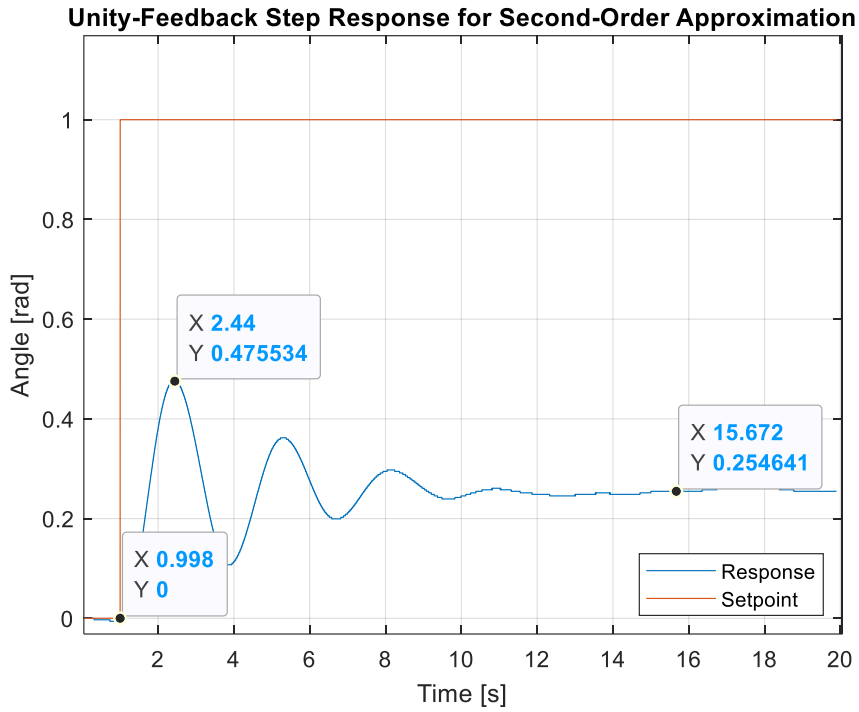


Figure 12: Unity-Feedback Step Response with Selected Data Points for Section 4.1

To calculate the PID gains, formulas given in the background section of [1] were used. These were:

$$k_d = \frac{2\zeta\omega_n + p_0 - b}{a} \tag{14}$$

$$k_p = \frac{{\omega_n}^2 + 2\zeta\omega_n p_0 - c}{a} \tag{15}$$

$$k_i = \frac{{\omega_n}^2 p_0}{a} \tag{16}$$

From Section 4.1 of the lab manual [1], the transfer function below was used:

$$G(s) = \frac{Y(s)}{R(s)} = \frac{y_{ss}\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2(1 - y_{ss})} = \frac{1.27332}{s^2 + 0.10047s + 3.62403} \tag{17}$$

Where:

$$G(s) = \frac{a}{s^2 + bs + c} \tag{18}$$

Therefore $a = 1.27332, b = 0.10047, c = 3.62403$. Given that the critical damping requires $\zeta = 1$, $\omega_n = 2\,rad/s$. The zero position $p_0 = 1$ was also used as instructed in the lab manual [1]. By plugging these values into equations 14, 15 and 16, we obtain the PID gain values:

$$k_d = \frac{2 * 1 * 2 + 1 - 0.10047}{1.27332} = 3.8471 \tag{19}$$

$$k_p = \frac{2^2 + 2 * 1 * 2 * 1 - 3.62403}{1.27332} = 3.4368 \tag{20}$$

$$k_i = \frac{2^2 * 1}{1.27332} = 3.1414 \tag{21}$$

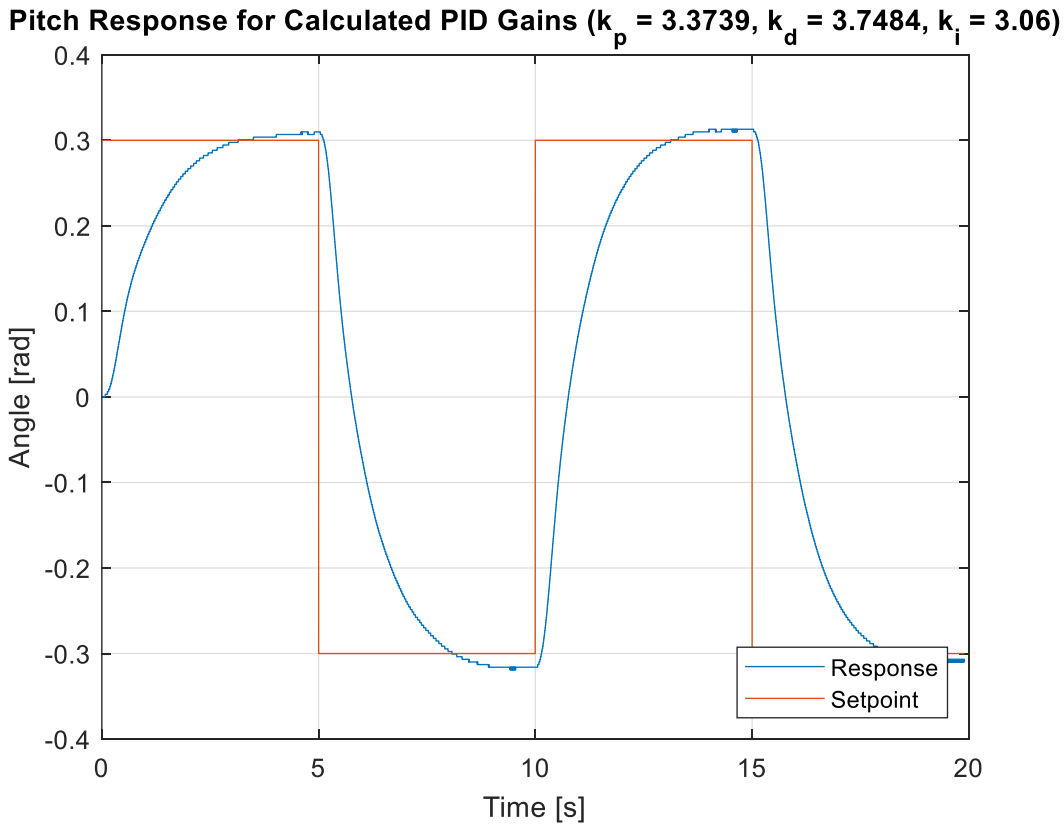


Figure 13: Response for Calculated PID Gains ($k_p = 3.3739, k_d = 3.7484, k_i = 3.06$)

Table 3: Measured Values from the Unity-Feedback Step Response Used for Second-Order Approximation (Section 4.1, Step 4)

Description	Symbol	Value	Units
Step Start Time	t_0	0.998	s
Maximum Response Time	t_{peak}	2.44	s
Peak Time	t_p	1.442	s
Maximum Response	y_{peak}	0.4755	rad
Steady-state Value	y_f	0.2546	rad
Percentage Overshoot	PO	86.75	%

Table 4: Calculated PID Gains from the Second-Order Approximation Model (Section 4.2, Step 1)

Description	Symbol	Value	Units
Tuned proportional gain	k_p	3.4368	-
Tuned derivative gain	k_d	3.8471	-
Tuned integral gain	k_i	3.1414	-

For implementation and testing in Section 4.2, the workbook solution values $k_p = 3.3739$, $k_d = 3.7484$, and $k_i = 3.06$were used, as shown in Figure 13.

Table 5: Response Values for the Calculated PID Controller (Section 4.2, Step 5)

Description	Symbol	Value	Units
Step Start Time	t_0	0	s
Maximum Response Time	t_{peak}	4.64	s
Peak Time	t_p	4.64	s
Maximum Response	y_{peak}	0.3099	rad
Steady-state Value	y_f	0.3099	rad
Percentage Overshoot	PO	0	%

IV. Analysis

IV.1 Effect of Proportional Gain on System Response

Increasing the proportional gain decreases the peak time slightly but significantly increases the overshoot and oscillation amplitude. From Figure 3 the peak time decreases from approximately 1.17 s for kp = 5 to about 1.1 s for kp = 10, showing a faster response. However, the percent overshoot increases from about 20% to over 60% as the gain increases. In addition, the oscillations become larger and do not decay, indicating the system becomes increasingly underdamped and approaches instability at higher gains. This shows that while proportional control can improve response speed, it cannot satisfy both the peak time and overshoot requirements simultaneously, making it impractical on its own.

IV.2 Effect of Derivative Gain on System Response

Increasing the derivative gain increases the damping of the system and eliminates oscillations but it also slows down the response. From Figure 4, all responses show zero percent overshoot, indicating that the system is highly damped for all values of kd. However, the peak time increases from approximately 1.6 s for kd = 4 to about 2.6 s for kd = 8, showing that higher derivative gain results in a slower system response. The system smoothly approaches the steady state value without oscillations, confirming that derivative control improves stability. This shows that while derivative control is effective at reducing overshoot and stabilizing the system, it negatively impacts response speed when used alone.

IV.3 Effect of Integral Gain on System Response

Increasing the integral gain reduces the steady state error but makes the response worse in terms of overshoot and oscillations. From Figure 7, when ki = 3 the system does not fully reach the setpoint and stays slightly below it, while for ki = 6 the system tracks the setpoint almost exactly with nearly zero steady state error. As ki increases the overshoot becomes much larger, with the response going up to about 0.4 rad for ki = 6. The oscillations also become more noticeable when the input changes between levels of the square wave. This shows that integral control helps eliminate steady state error but increasing it too much makes the system less stable and increases oscillations.

IV.4 Controller Tuning and Improvement of System Response

The response of the system can be improved by combining proportional, derivative and integral control instead of using each one separately. The controller was tuned to approximately kp = 8, kd = 3.1 and ki = 5 to balance speed, stability and accuracy. These values were chosen by adjusting each gain while observing how the system response changed until a good tradeoff between fast response, low overshoot and small steady state error was achieved. The proportional gain was set high enough to make the system respond quickly while the derivative gain was used to reduce oscillations and smooth out the response. The integral gain was then increased to remove the remaining steady state error and improve tracking of the input. With this combination the system was able to follow the input closely, settle in a reasonable amount of time, and avoid the large oscillations seen when using individual control actions.

IV.5 Evaluation of the Second Order Approximation Model

The data from the plant response and the second order approximation were obtained from two separate simulations. The plant response was taken from the experimental data, while the second order model response was generated using the approximated transfer function. These two responses were then plotted together on the same figure to allow direct comparison of their dynamic behavior.

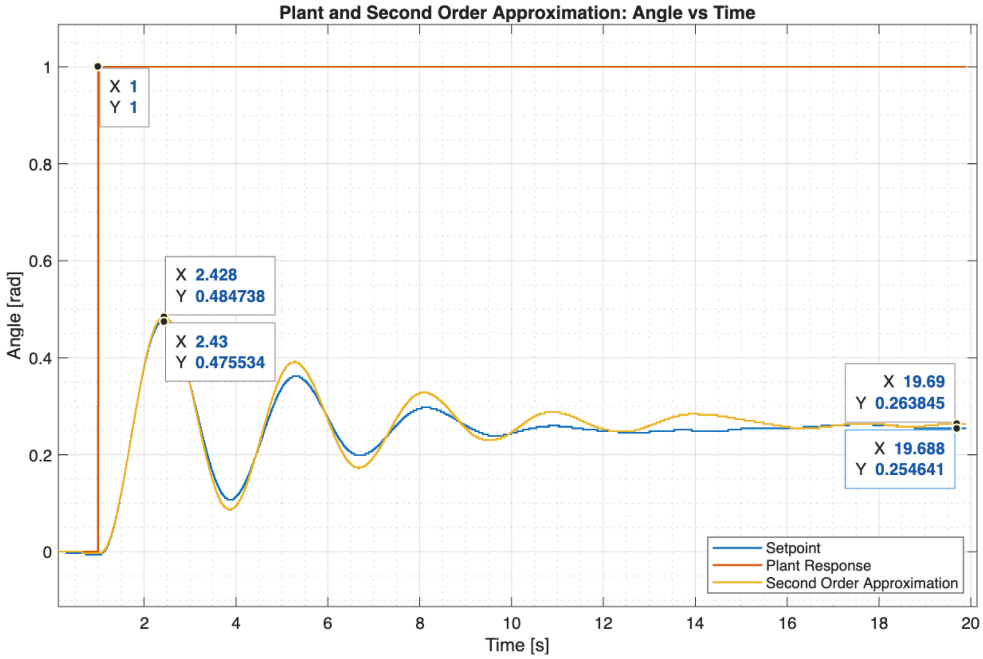


Figure 14: Combined Data of model experiment and 2nd order approximation

From this combined plot, key points were selected for both responses. These include the step start time, the first peak value and its corresponding time, and the steady state value. These points were used to calculate peak time, percent overshoot, and steady state error for both the plant and the model.

Peak time plant:

$$t_p = t_{peak} - t_0 = 2.43 - 1.00 = 1.43\,s \tag{22}$$

Peak time model

$$t_p = t_{peak} - t_0 = 2.428 - 1.00 = 1.428\,s \tag{23}$$

Percent overshoot plant:

$$PO_{plant} = \left(\frac{y_{peak} - y_{ss}}{y_{ss}}\right) \times 100 = \left(\frac{0.475534 - 0.254641}{0.254641}\right) \times 100 = 86.75\,\% \tag{24}$$

Percent overshoot model:

$$PO_{model} = \left(\frac{y_{peak} - y_{ss}}{y_{ss}}\right) \times 100 = \left(\frac{0.484738 - 0.263845}{0.263845}\right) \times 100 = 83.73\,\% \tag{25}$$

Plant-model steady state error difference:

$$e_{ss} = |y_{ss}, model - y_{ss}, plant| = |0.263845 - 0.254641| = 0.0092\,rad \tag{26}$$

$$e_{ss},\% = \left(\frac{0.0092}{0.254641}\right) \times 100 = 3.61\,\% \tag{27}$$

Peak time & PO percent error:

$$e_{tp} = \left|\frac{t_p, model - t_p, plant}{t_p}\right| \times 100 = \left|\frac{1.428 - 1.43}{1.43}\right| \times 100 = 0.14\,\% \tag{28}$$

$$e_{PO} = \left|\frac{PO_{model} - PO_{plant}}{PO_{plant}}\right| \times 100 = \left|\frac{83.73 - 86.75}{86.75}\right| \times 100 = 3.48\,\% \tag{29}$$

The second order approximation matches the real system pretty well in terms of timing and overall shape of the response. The peak time error is very small at 0.14%, which means the model predicts how fast the system responds quite accurately. The percent overshoot and steady state values also have relatively small errors of around 3 to 4%, so the model is doing a decent job at capturing the system behavior. The model slightly overestimates the steady state value and does not match the oscillation amplitude exactly. This is likely because the real system has effects like friction, actuator limits, and other higher order dynamics that are not included in the second order model. So overall, the approximation is good enough for general predictions and controller design, but it is not fully accurate compared to the real system.

IV.6 Comparison of Tuned Controller and Calculated Controller Response

The manually tuned controller gives a better response than the one using the calculated gains. With kp = 8, kd = 3.1, and ki = 5, the system follows the square wave input closely and reacts quickly when the input changes. The response rises fast and reaches the setpoint with only small overshoot around 0.32 to 0.33 rad for a 0.3 rad input. It also settles quickly and stays close to the desired value, so the tracking is good overall.

The response using the calculated gains is slower. The system takes more time to reach the setpoint and the rise and fall are more gradual. Even though the overshoot is small and the response looks smooth, it does not follow the square wave very well because it reacts too slowly. Because of this, the calculated controller does not fully meet the requirements, while the tuned controller gives a better and more accurate response.

V. Home Analysis

V.1 Bode Plot Comparison

The Bode plots were obtained by first defining the second order transfer function of the Quanser Aero model using the values found from the second order approximation. This transfer function was used to generate the open loop Bode plot of the plant. After that, the PID controller with the calculated gains was defined and multiplied by the plant transfer function to form the compensated open loop system.

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \tag{30}$$

$$(s) = k_p + \frac{k_i}{s} + k_d s \tag{31}$$

$$CL(s) = C(s)G(s) = \left(k_p + \frac{k_i}{s} + k_d s\right)\left(\frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}\right) \tag{32}$$

MATLAB was then used to generate the Bode plots and margin values for both cases using the margin() function. In this way, the frequency response of the original plant and the controlled system could be compared directly.

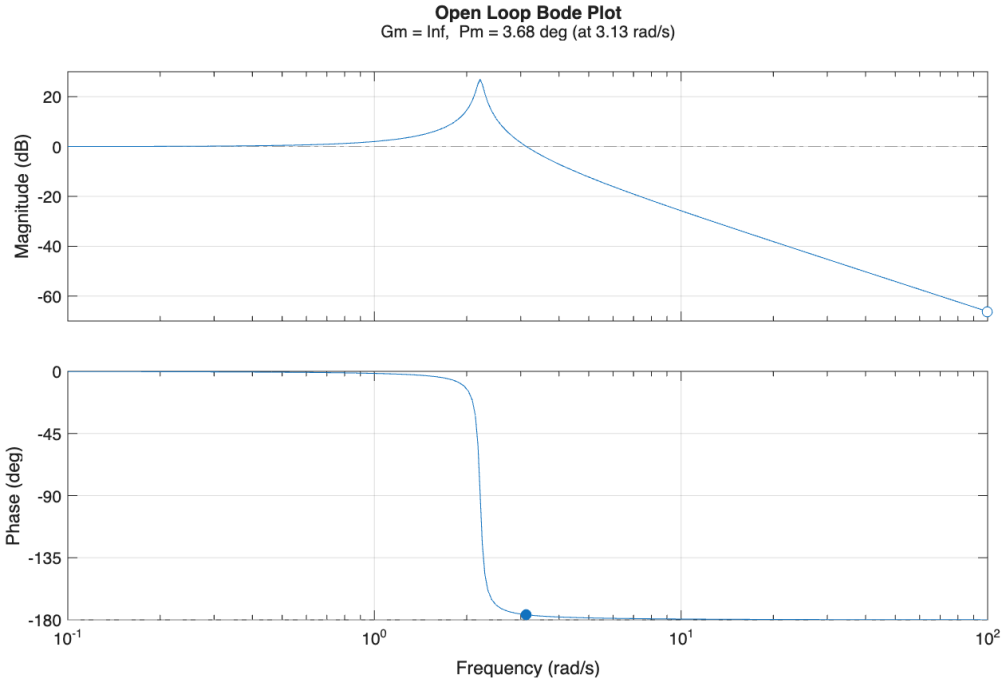


Figure 15: Open Loop Bode Plot

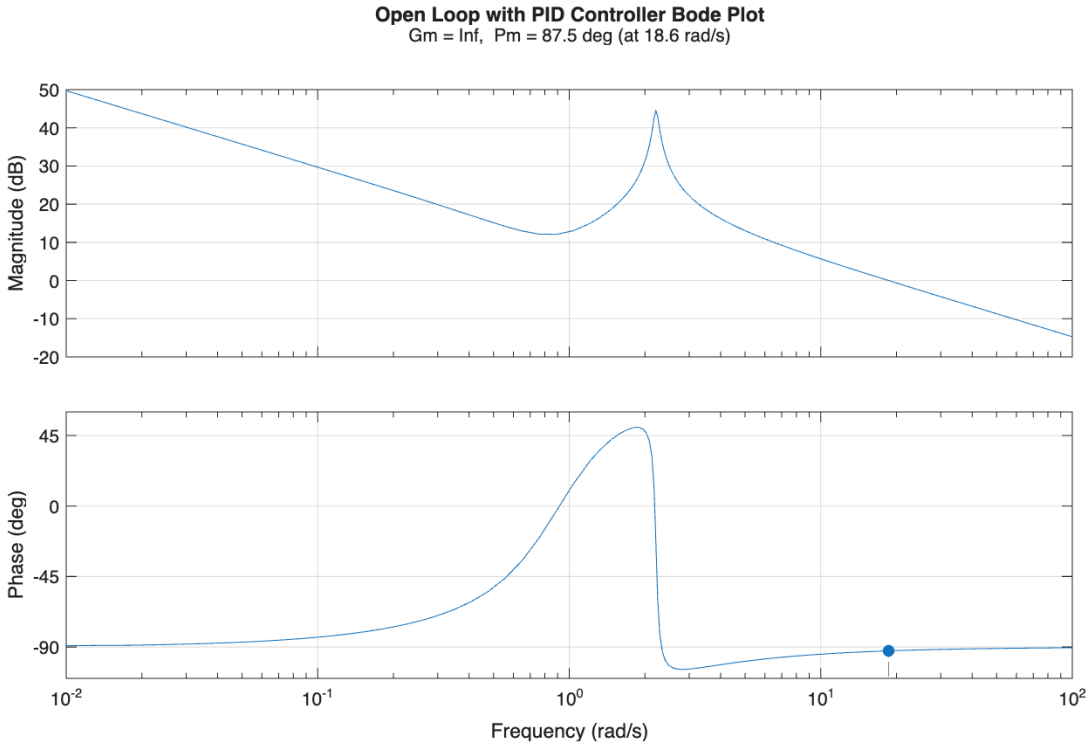


Figure 16: Open Loop with PID Bode Plot

The open loop Bode plot shows that the original plant has a very small phase margin of about 3.68° while the gain margin is infinite. This means the system is barely stable and very close to instability and will have strong oscillations. After adding the PID controller, the phase margin increases to about 87.5°, which is a big improvement. The system becomes much more stable and better damped. Overall, the controller makes the system more stable and improves how it responds.

V.2 Phase Margin, Gain Margin, and Bandwidth Results

Table 6: Phase Margin, Gain Margin & Bandwidth Result Table

System	Phase Margin [deg]	Gain Margin	Bandwidth [rad/s]
Open Loop	3.68	∞	N/A
PID Controlled	87.5	∞	19.32

V.3 State Space Model Derivation

Start with the transfer function:

$$G(s) = \frac{Y(s)}{U(s)} = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \tag{33}$$

$$(s^2 + 2\zeta\omega_n s + \omega_n^2)Y(s) = \omega_n^2 U(s) \tag{34}$$

Time domain:

$$y'' + 2\zeta\omega_n y' + \omega_n^2 y = \omega_n^2 u \tag{35}$$

Defining states:

$$x_1 = y \tag{36}$$

$$x_2 = y' \tag{37}$$

Then:

$$x_1' = x_2 \tag{38}$$

$$x_2' = -2\zeta\omega_n x_2 - \omega_n^2 x_1 + \omega_n^2 u \tag{39}$$

State Space Form

$$x' = Ax + Bu \tag{40}$$

$$y = Cx + Du \tag{41}$$

$$\begin{bmatrix} 0 & 1 \\ -\omega n^2 & -2\zeta\omega n \end{bmatrix} \tag{42}$$

$$\begin{bmatrix} 0 \\ \omega n^2 \end{bmatrix} \tag{43}$$

$$[1 \quad 0] \tag{44}$$

$$[0] \tag{45}$$

V.4 Observability and Controllability Matrix Derivation

$$P_c = [B \quad AB] \tag{46}$$

$$AB = \begin{bmatrix} (0 \cdot 0) + (1 \cdot \omega n^2) \\ (-\omega n^2 \cdot 0) + ((-2\zeta\omega n) \cdot \omega n^2) \end{bmatrix} = \begin{bmatrix} \omega n^2 \\ -2\zeta\omega^2 n^3 \end{bmatrix} \tag{47}$$

$$P_c = \begin{bmatrix} 0 & \omega n^2 \\ \omega n^2 & -2\zeta\omega^2 n^3 \end{bmatrix} \tag{48}$$

$$P_o = \begin{bmatrix} C \\ CA \end{bmatrix} \tag{49}$$

$$CA = [(1)(0) + (0)(-\omega n^2) \quad (1)(1) + (0)(-2\zeta\omega n)] = [0 \quad 1] \tag{50}$$

$$P_o = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \tag{51}$$

V.5 Results for Controllability and Observability

$$P_c = \begin{bmatrix} 0 & \omega n^2 \\ \omega n^2 & -2\zeta\omega^2 n^3 \end{bmatrix} \tag{52}$$

$$\det(P_c) = 0 \cdot (-2\zeta\omega_n^3) - (\omega_n^2)(\omega_n^2) = -\omega_n^4 \neq 0 \tag{53}$$

$$rank(P_c) = 2 \tag{54}$$

$$P_o = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \tag{55}$$

$$det(P_o) = 1 \cdot 1 - 0 \cdot 0 = 1 \neq 0 \tag{56}$$

$$rank(P_o) = 2 \tag{57}$$

The system is observable and controllable.

V.5 Results for Controllability and Observability

The state space control approach works for this system because it is both controllable and observable so the states can be controlled and estimated. For this system it is not the best choice because it is more complex than needed. The system is simple and already works well with a PID controller, which is easier to implement and tune. State space control would require a full model and possibly an observer which adds unnecessary complexity. So, while it works, it is not the most practical approach for this system.

VI. Conclusions

This final project investigated the response of the Quanser Aero under proportional, derivative and integral control and compared manual PID tuning with a controller designed from a second-order system approximation. The unity feedback response showed that the uncompensated system was highly underdamped. For that system, the peak time was 1.694 seconds, a 5% settling time of 13.63 seconds and a percentage overshoot of 68.98%. These parameters confirmed that compensation was required. Varying the proportional gain increased response speed but it also increased overshoot and oscillations. Increasing the derivative gain improved but it slowed the response. Increasing the integral gain reduced steady-state error but larger values also increased overshoot. After combining all three controls, the tuned controller with $k_p = 8$, $k_d = 3.1$ and $k_i = 5$ produced the best overall response. The peak time was 1.18 seconds, percentage overshoot was 4.12% and a steady-state value was 0.2976 radians, which was very close to the required 0.3 radians.

This experiment also showed that a second-order approximation could be used to model the Quanser Aero and to analytically calculate the PID gains. The approximated model captured the system behavior well and had a small peak-time and overshoot errors relative to the plant. But, the calculated controller produced a slower response than the manually tuned controller. Although it's overshoot was small, it didn't track the square-wave command as effectively and didn't satisfy the desired response requirements as well as the tuned controller. This makes it so that the overall goals of the experiment were only partially accomplished. The goal of understanding the effects of PID gains, building a second-order approximation and analytically designing the controller were accomplished and the manually tuned controller met the performance goals. What wasn't accomplished is the calculated controller providing the same level performance. To improve the results, a more accurate model of the Quanser Aero could be developed by including high-order dynamics and non-linear behaviour.

VII. References

[1] Moncayo H. Ph.D. Student Workbook Quanser AERO – FINAL PROJECT. Embry-Riddle Aeronautical University. 2023.

[2] Quanser Inc. QUARC User Manual

VIII. Appendix

Table of Contents

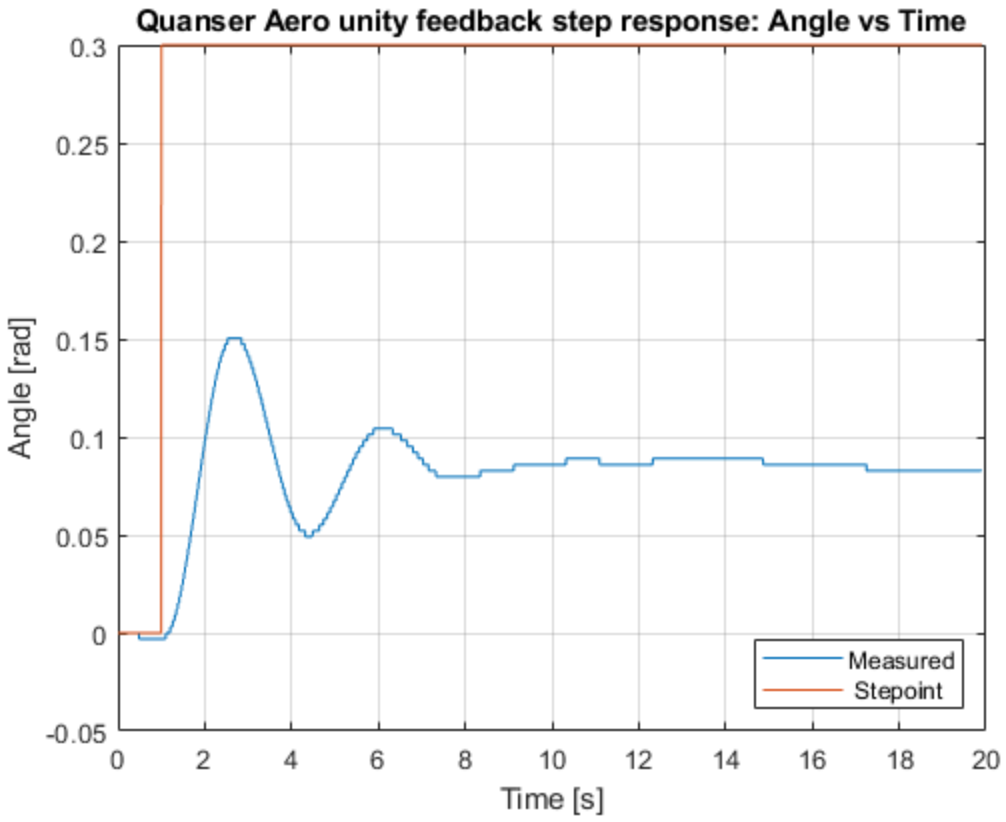
..... 1
2.1 Open Loop Response 1
2.2 Proportional Control 3
2.2 Proportional Gain: Angle 5
2.2 Proportional Gain: Voltage kp=5 6
2.2 Proportional Gain: Voltage kp-7.5 7
2.2 Proportional Gain: Voltage kp-10 8
2.3 Derivative Control: Angle 9
2.3 Derivative Gain: Voltage kd=4 10
2.3 Derivative Gain: Voltage kd=6 11
2.3 Derivative Gain: Voltage kd=8 12
2.3 Derivative Control: kd < 0.25 13
2.4 Integral Control: Test, d_3 14
2.4 Integral Control: Angle 15
2.4 Integral Gain: Voltage ki=3 16
2.4 Integral Gain: Voltage ki=4.5 17
2.4 Integral Gain: Voltage ki=6 18
2.4 Integral Control, Step 3, ki=5 19
2.5 Response Tuning 21
4.1 Second-order appox, Step 3 23
Custom Transfer Function 25
Tuned sq wave 27

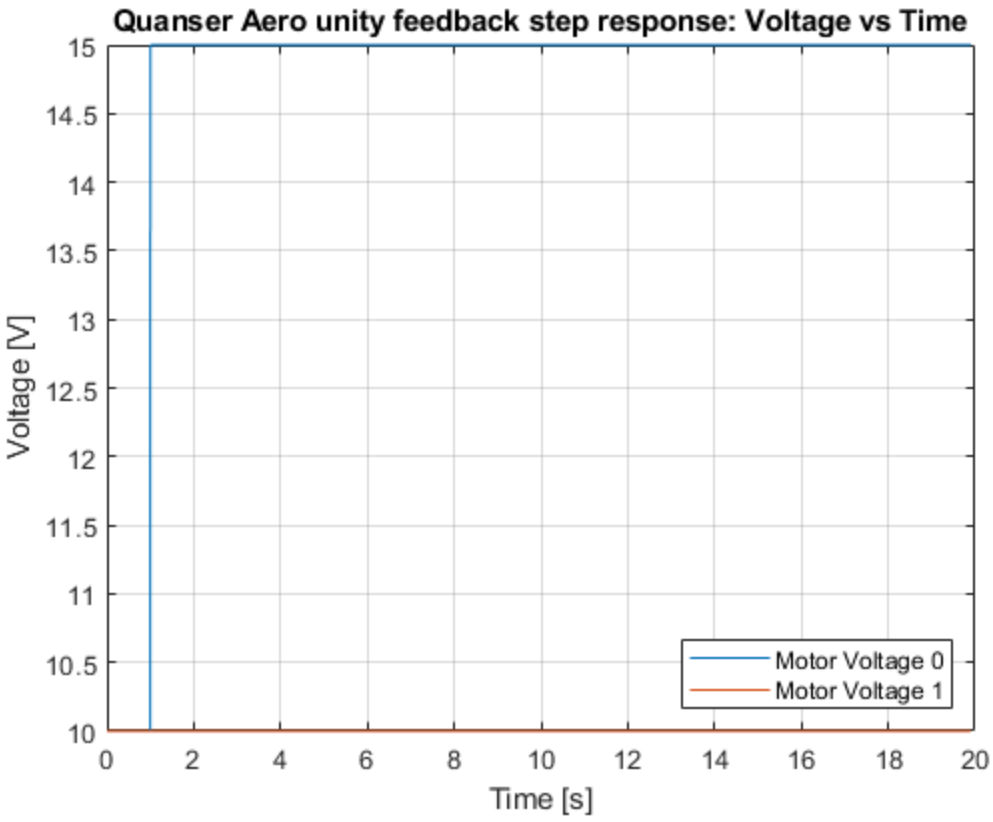

```
% Authors: Anurananan Bharadwaj, Kalkamanali Satvaldy, Dobromir Sokolowski
% AE443 Final Project - Quanser Aero
```

2.1 Open Loop Response

```
clear
clc
close all

load("First_Test.mat")
figure(1)
plot(Angle(:,1), Angle(:,2))
hold on
plot(Angle(:,1), Angle(:,3)) % Setpoint
grid on
xlabel("Time [s]")
ylabel("Angle [rad]")
title("Quanser Aero unity feedback step response: Angle vs Time")
legend("Measured"," Stepoint", "Location", "SouthEast")

figure(2)
plot(Voltage(:,1), Voltage(:,2))
hold on
plot(Voltage(:,1), Voltage(:,3)) % setpoint
```

```
grid on
xlabel("Time [s]")
ylabel("Voltage [V]")
title("Quanser Aero unity feedback step response: Voltage vs Time")
legend("Motor Voltage 0", "Motor Voltage 1","Location", "SouthEast")
```

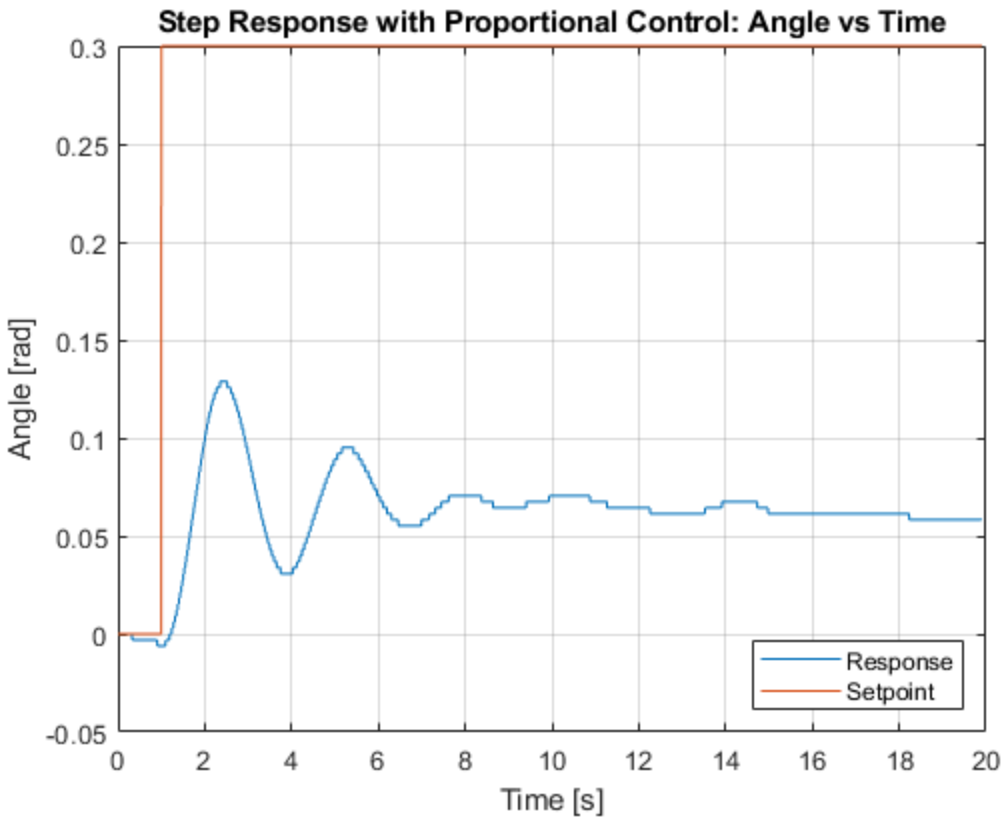


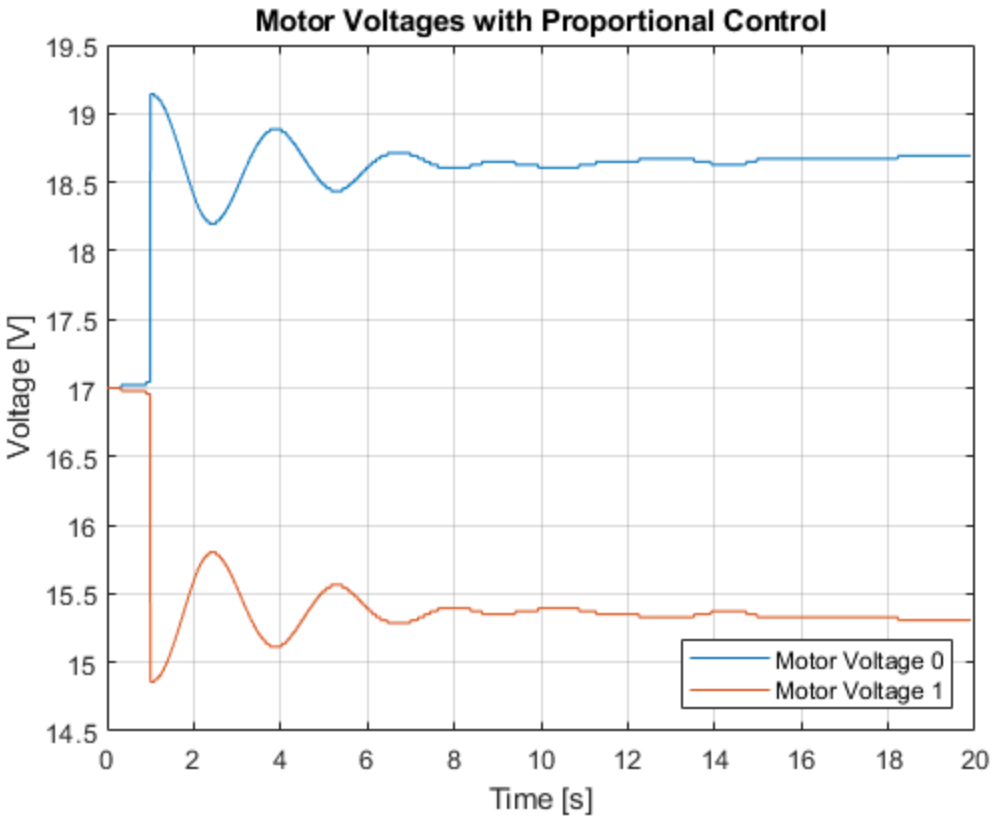


2.2 Proportional Control

```
clear
clc
close all

load("Second_Test.mat")
figure(3)
plot(Angle(:,1), Angle(:,2)) %
hold on
plot(Angle(:,1), Angle(:,3)) %setpoint
grid on
xlabel("Time [s]")
ylabel("Angle [rad]")
title("Step Response with Proportional Control: Angle vs Time")
legend("Response", "Setpoint", "Location", "SouthEast")

figure(4)
plot(Voltage(:,1), Voltage(:,2))
hold on
plot(Voltage(:,1), Voltage(:,3))  % setpoint
grid on
xlabel("Time [s]")
ylabel("Voltage [V]")
title("Motor Voltages with Proportional Control")
```

```
legend("Motor Voltage 0", "Motor Voltage 1","Location", "SouthEast")
```

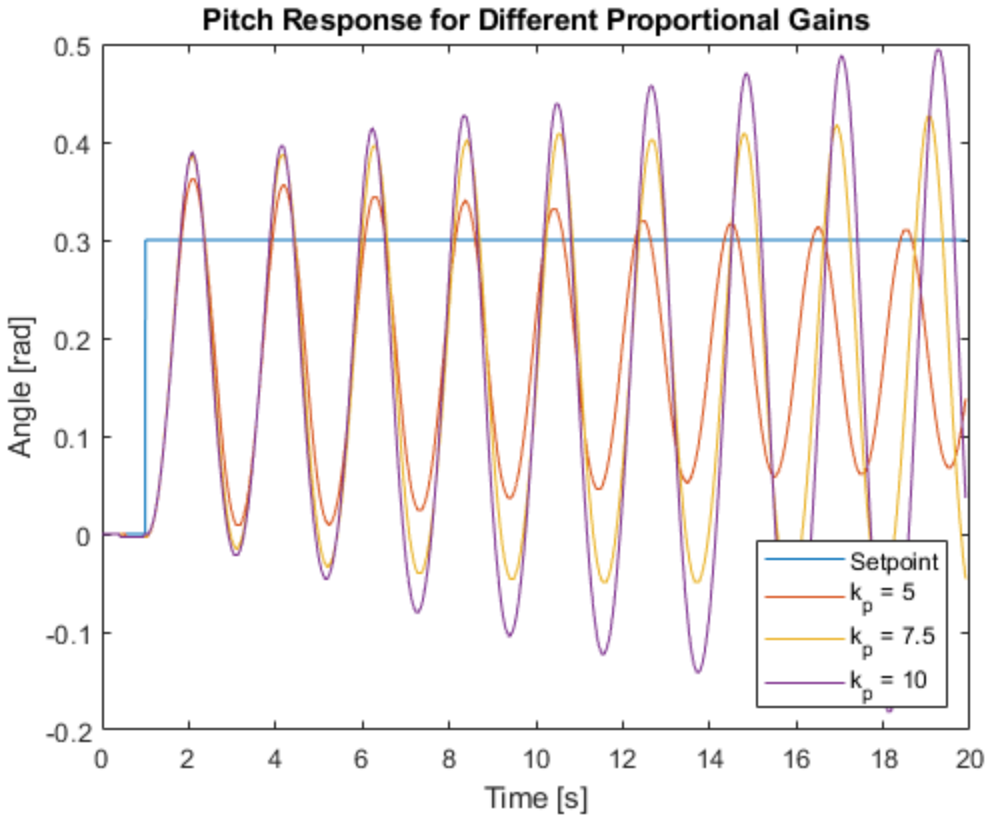




2.2 Proportional Gain: Angle

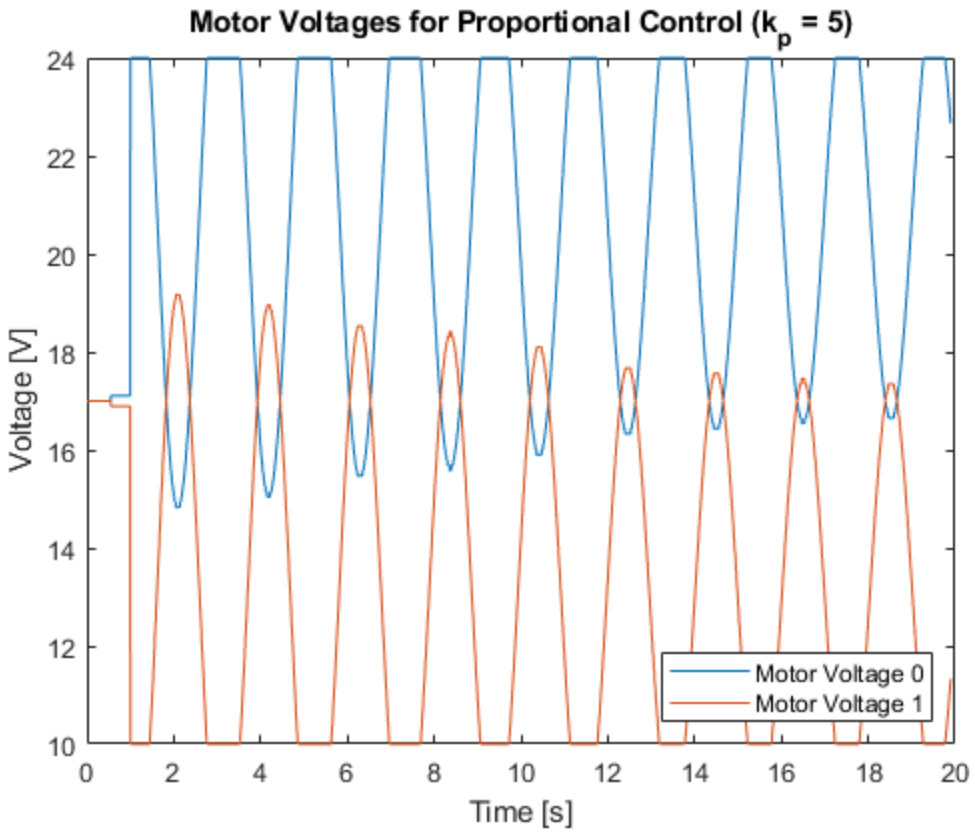
```
clear
clc
close all

figure(5)
load("third_Test_gain_5.mat")
grid on
plot(Angle(:,1), Angle(:,3))
hold on
plot(Angle(:,1), Angle(:,2))
hold on
load("4_Test_gain75.mat")
plot(Angle(:,1), Angle(:,2))
hold on
hold on
load("5_Test_gain10.mat")
plot(Angle(:,1), Angle(:,2))
hold on
hold on
xlabel("Time [s]")
ylabel("Angle [rad]")
title("Pitch Response for Different Proportional Gains")
```

```
legend("Setpoint", "k_p = 5", "k_p = 7.5", "k_p =
 10", "Location", "SouthEast")
```



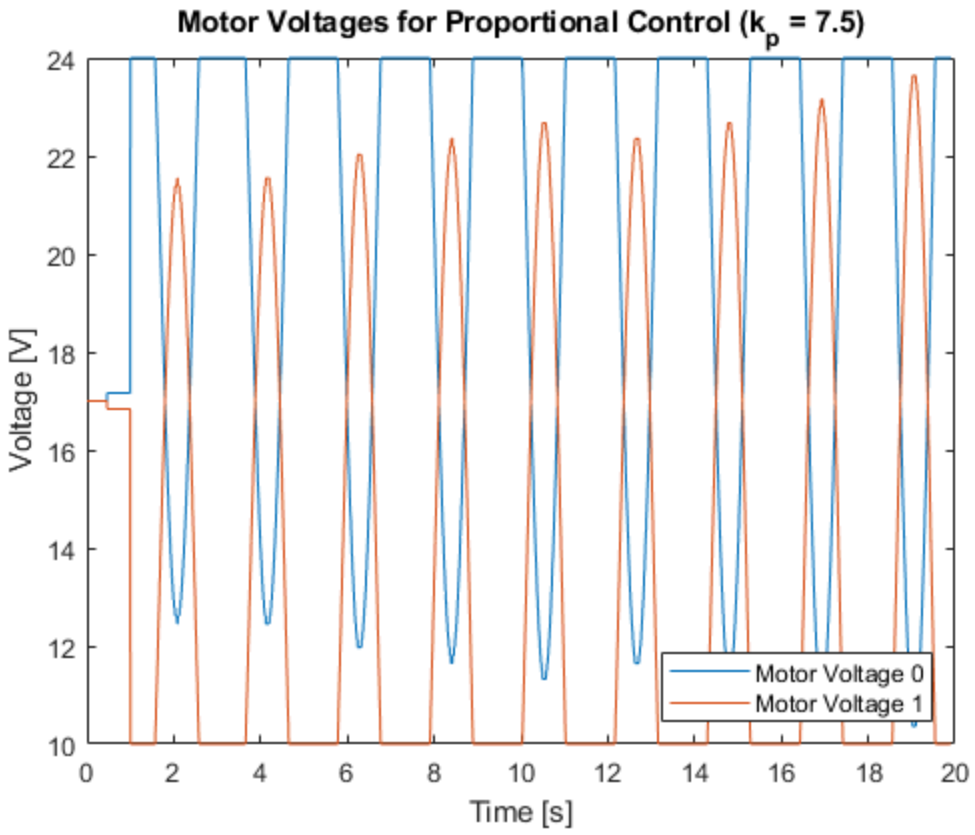
2.2 Proportional Gain: Voltage kp=5

```
clear
clc
close all

figure(6)
load("third_Test_gain_5.mat")
grid on
plot(Voltage(:,1), Voltage(:,2))
hold on
plot(Voltage(:,1), Voltage(:,3))

xlabel("Time [s]")
ylabel("Voltage [V]")
title("Motor Voltages for Proportional Control (k_p = 5)")
legend("Motor Voltage 0", "Motor Voltage 1", "Location", "SouthEast")
```



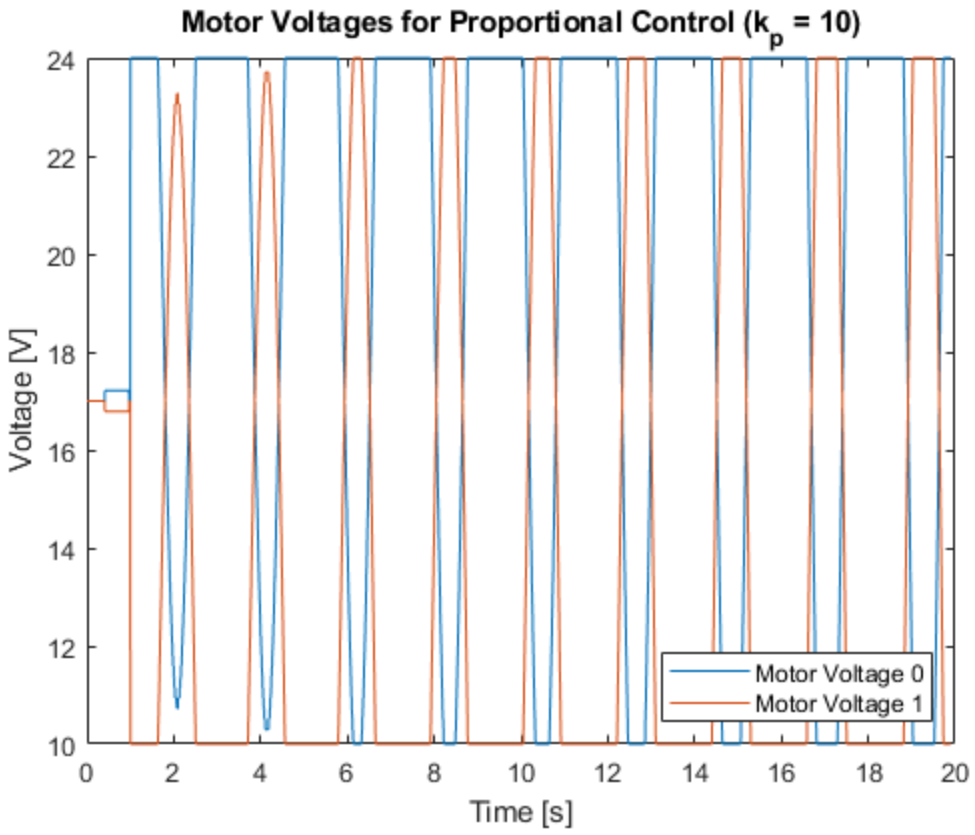
2.2 Proportional Gain: Voltage kp-7.5

```
clear
clc
close all

figure(7)
load("4_Test_gain75.mat")
grid on
plot(Voltage(:,1), Voltage(:,2))
hold on
plot(Voltage(:,1), Voltage(:,3))
hold on

xlabel("Time [s]")
ylabel("Voltage [V]")
title("Motor Voltages for Proportional Control (k_p = 7.5)")
legend("Motor Voltage 0", "Motor Voltage 1", "Location", "SouthEast")
```



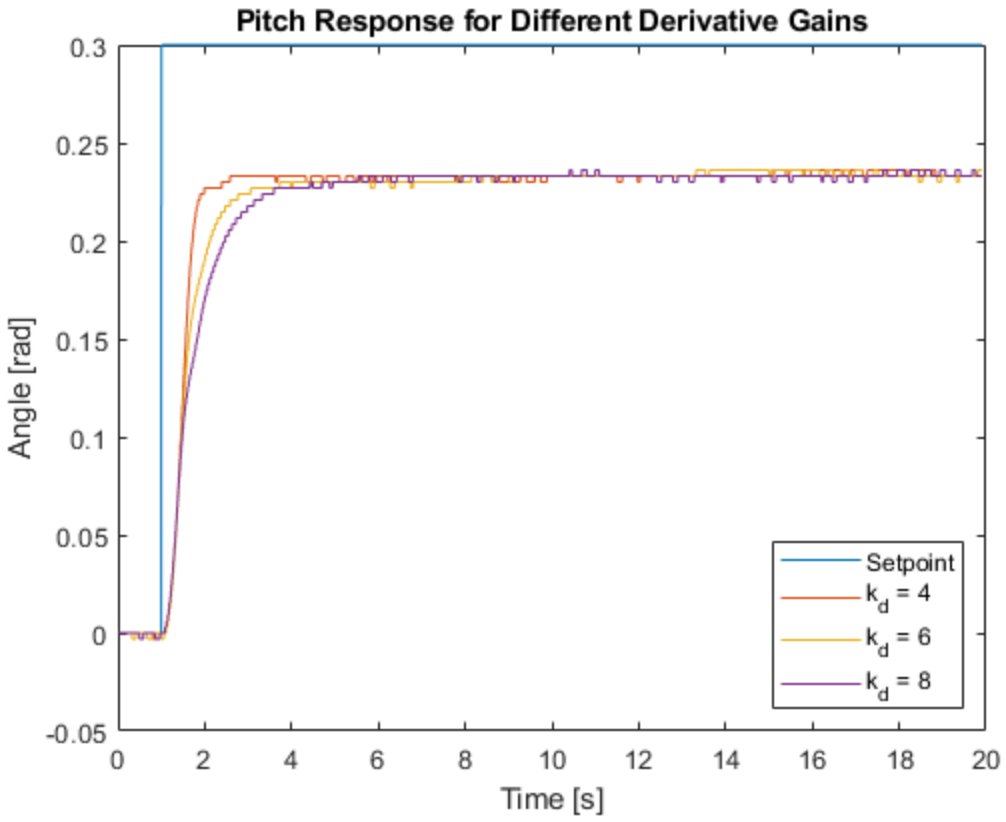
2.2 Proportional Gain: Voltage kp-10

```
clear
clc
close all

figure(8)
load("5_Test_gain10.mat")
plot(Voltage(:,1), Voltage(:,2))
hold on
plot(Voltage(:,1), Voltage(:,3))
hold on

xlabel("Time [s]")
ylabel("Voltage [V]")
title("Motor Voltages for Proportional Control (k_p = 10)")
legend("Motor Voltage 0", "Motor Voltage 1", "Location", "SouthEast")
```



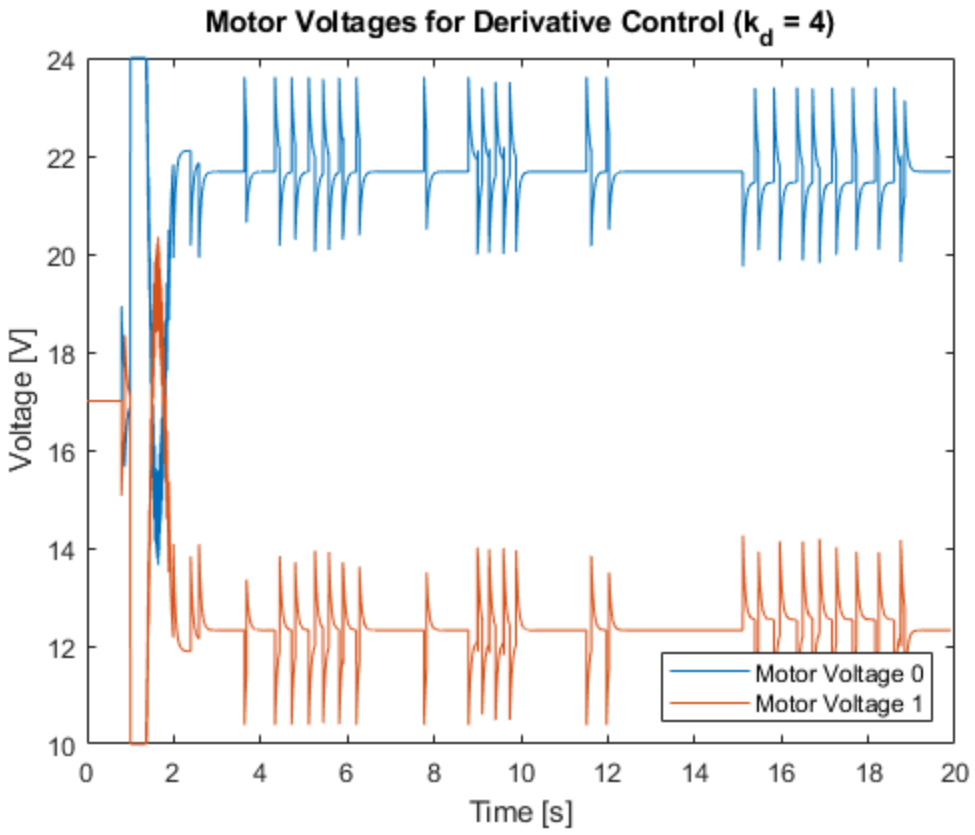
2.3 Derivative Control: Angle

```
clear
clc
close all

figure(9)
load("6_Test_gain_d_4.mat")
grid on
plot(Angle(:,1), Angle(:,3))
hold on
plot(Angle(:,1), Angle(:,2))
hold on
load("7_Test_gain_d_6.mat")
plot(Angle(:,1), Angle(:,2))
hold on
hold on
load("8_Test_gain_d_8.mat")
plot(Angle(:,1), Angle(:,2))
hold on
hold on
xlabel("Time [s]")
ylabel("Angle [rad]")
title("Pitch Response for Different Derivative Gains")
legend("Setpoint", "k_d = 4", "k_d = 6", "k_d = 8", "Location", "SouthEast")
```



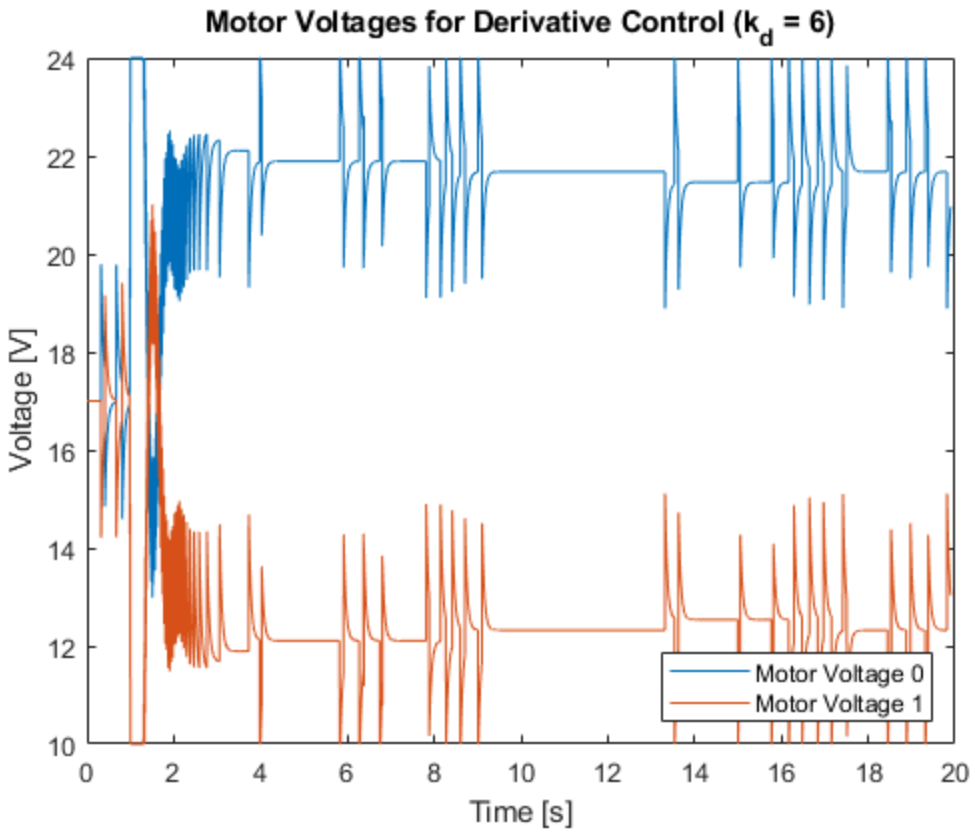
2.3 Derivative Gain: Voltage kd=4

```
clear
clc
close all

figure(10)
load("6_Test_gain_d_4.mat")
grid on
plot(Voltage(:,1), Voltage(:,2))
hold on
plot(Voltage(:,1), Voltage(:,3))

xlabel("Time [s]")
ylabel("Voltage [V]")
title("Motor Voltages for Derivative Control (k_d = 4)")
legend("Motor Voltage 0", "Motor Voltage 1", "Location", "SouthEast")
```



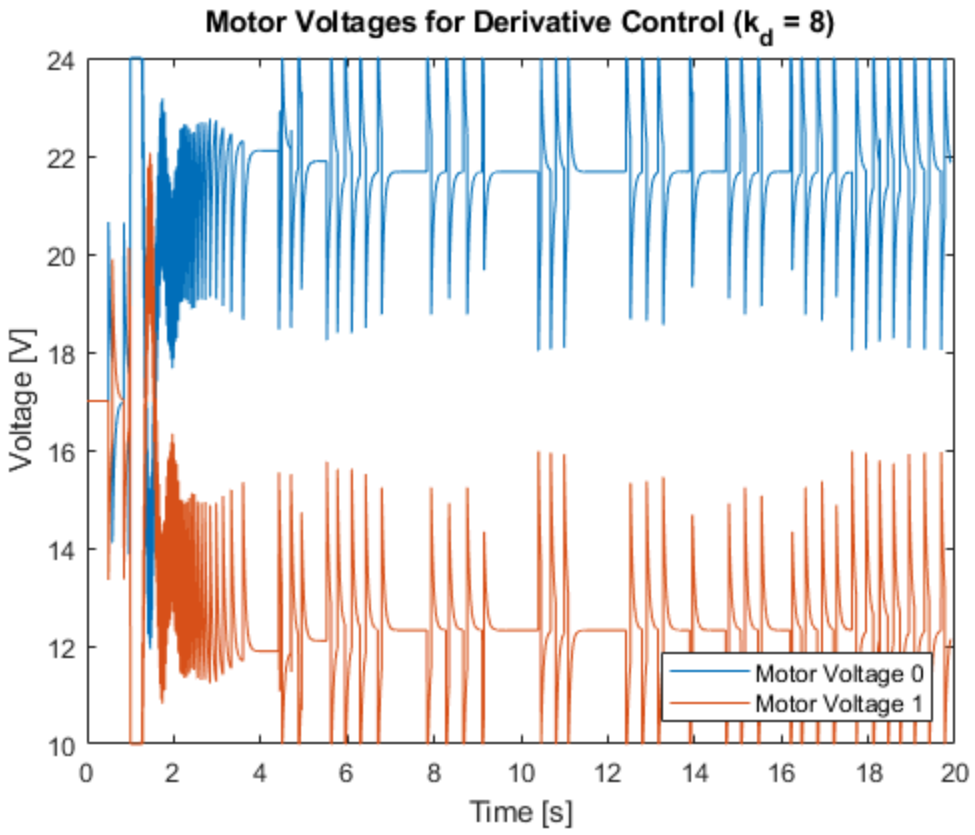
2.3 Derivative Gain: Voltage kd=6

```
clear
clc
close all

figure(11)
load("7_Test_gain_d_6.mat")
grid on
plot(Voltage(:,1), Voltage(:,2))
hold on
plot(Voltage(:,1), Voltage(:,3))
hold on

xlabel("Time [s]")
ylabel("Voltage [V]")
title("Motor Voltages for Derivative Control (k_d = 6)")
legend("Motor Voltage 0", "Motor Voltage 1", "Location", "SouthEast")
```



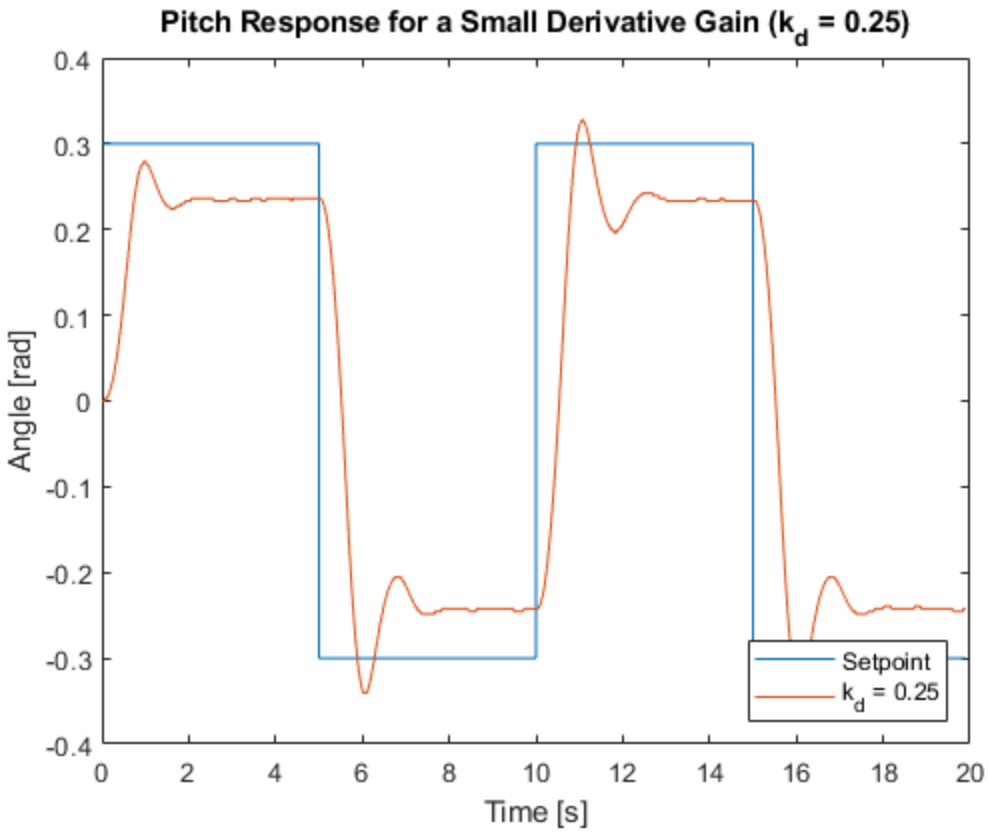
2.3 Derivative Gain: Voltage kd=8

```
clear
clc
close all

figure(12)
load("8_Test_gain_d_8.mat")
plot(Voltage(:,1), Voltage(:,2))
hold on
plot(Voltage(:,1), Voltage(:,3))
hold on

xlabel("Time [s]")
ylabel("Voltage [V]")
title("Motor Voltages for Derivative Control (k_d = 8)")
legend("Motor Voltage 0", "Motor Voltage 1", "Location", "SouthEast")
```



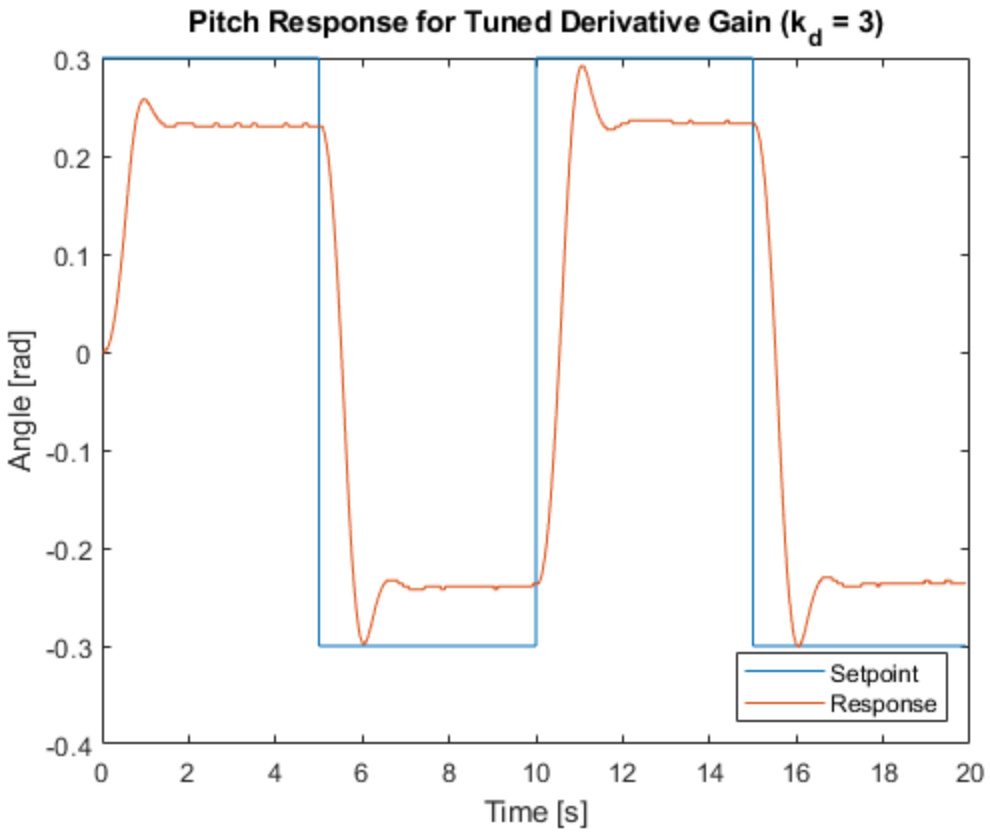
2.3 Derivative Control: kd < 0.25

```
clc
clear
close all

figure(13)
load("23_step3_gain_d_025.mat")
grid on
plot(ScopeData1(:,1), ScopeData1(:,3))
hold on
plot(ScopeData1(:,1), ScopeData1(:,2))
xlabel("Time [s]")
ylabel("Angle [rad]")
title("Pitch Response for a Small Derivative Gain (k_d = 0.25)")
legend("Setpoint", "k_d = 0.25", "Location", "SouthEast")
```



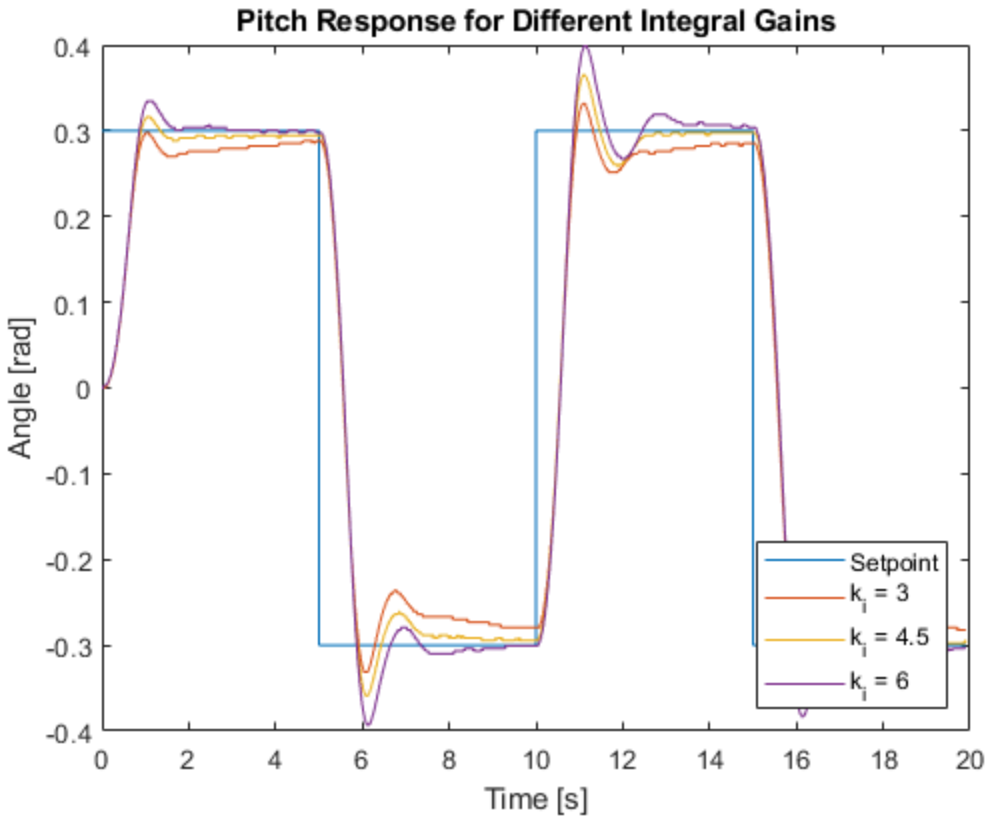
2.4 Integral Control: Test, d_3

```
clear
clc
close all

figure(13)
load("9__sq_wave_Test_gain_d_3.mat")
grid on
plot(Angle(:,1), Angle(:,3))
hold on
plot(Angle(:,1), Angle(:,2))

xlabel("Time [s]")
ylabel("Angle [rad]")
title("Pitch Response for Tuned Derivative Gain (k_d = 3)")
legend("Setpoint", "Response", "Location", "SouthEast")
```



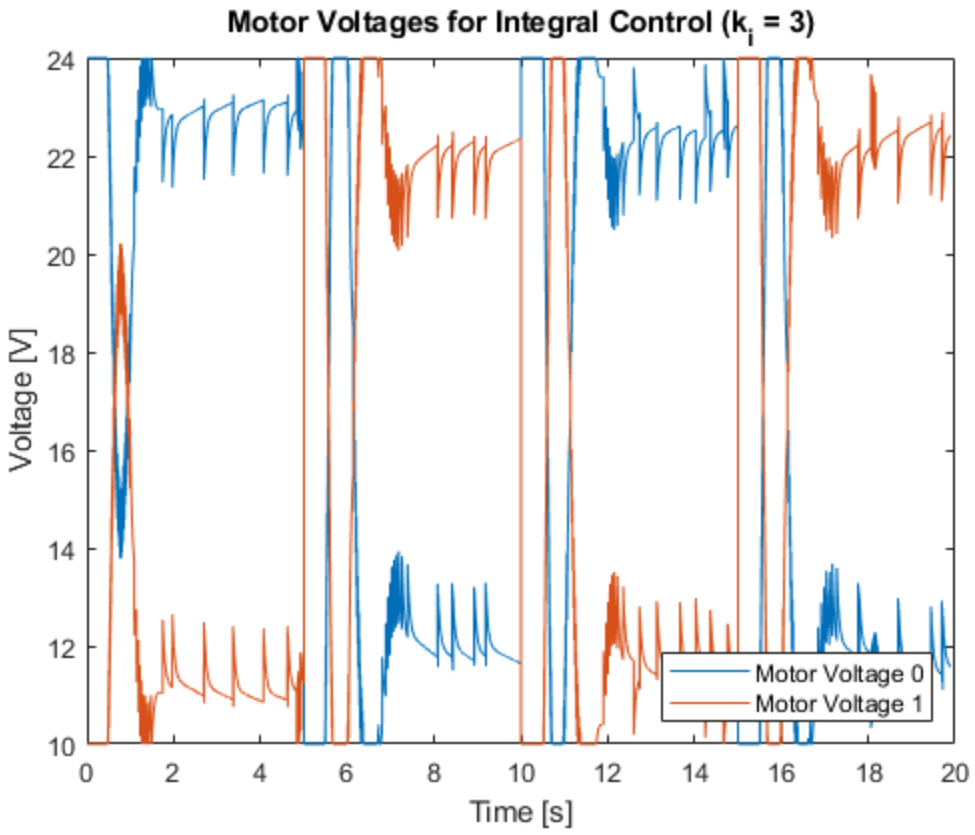
2.4 Integral Control: Angle

```
clc
clear
close all

figure(14)
load("10_sq_wave_gain_i_3.mat")
grid on
plot(Angle(:,1), Angle(:,3))
hold on
plot(Angle(:,1), Angle(:,2))
hold on
load("11_sq_wave_gain_i_45.mat")
plot(Angle(:,1), Angle(:,2))
hold on
hold on
load("12_sq_wave_gain_i_6.mat")
plot(Angle(:,1), Angle(:,2))
hold on
hold on
xlabel("Time [s]")
ylabel("Angle [rad]")
title("Pitch Response for Different Integral Gains")
legend("Setpoint", "k_i = 3", "k_i = 4.5", "k_i = 6", "Location", "SouthEast")
```



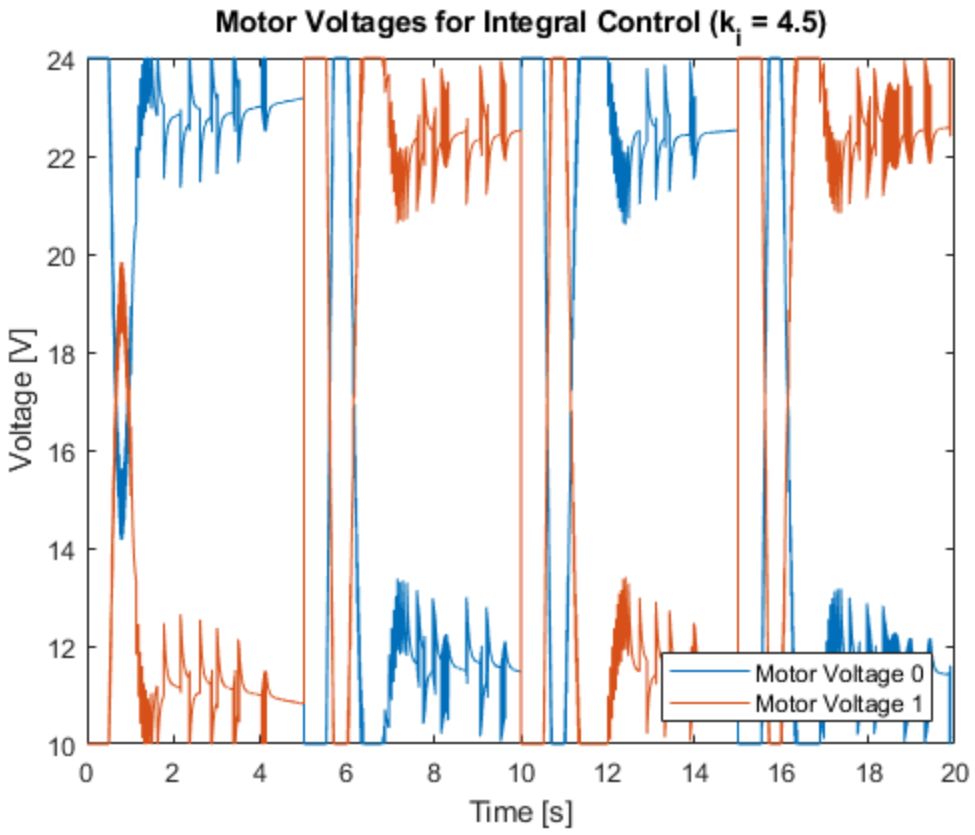
2.4 Integral Gain: Voltage ki=3

```
clear
clc
close all

figure(15)
load("10_sq_wave_gain_i_3.mat")
plot(Voltage(:,1), Voltage(:,2))
hold on
plot(Voltage(:,1), Voltage(:,3))
hold on

xlabel("Time [s]")
ylabel("Voltage [V]")
title("Motor Voltages for Integral Control (k_i = 3)")
legend("Motor Voltage 0", "Motor Voltage 1", "Location", "SouthEast")
```



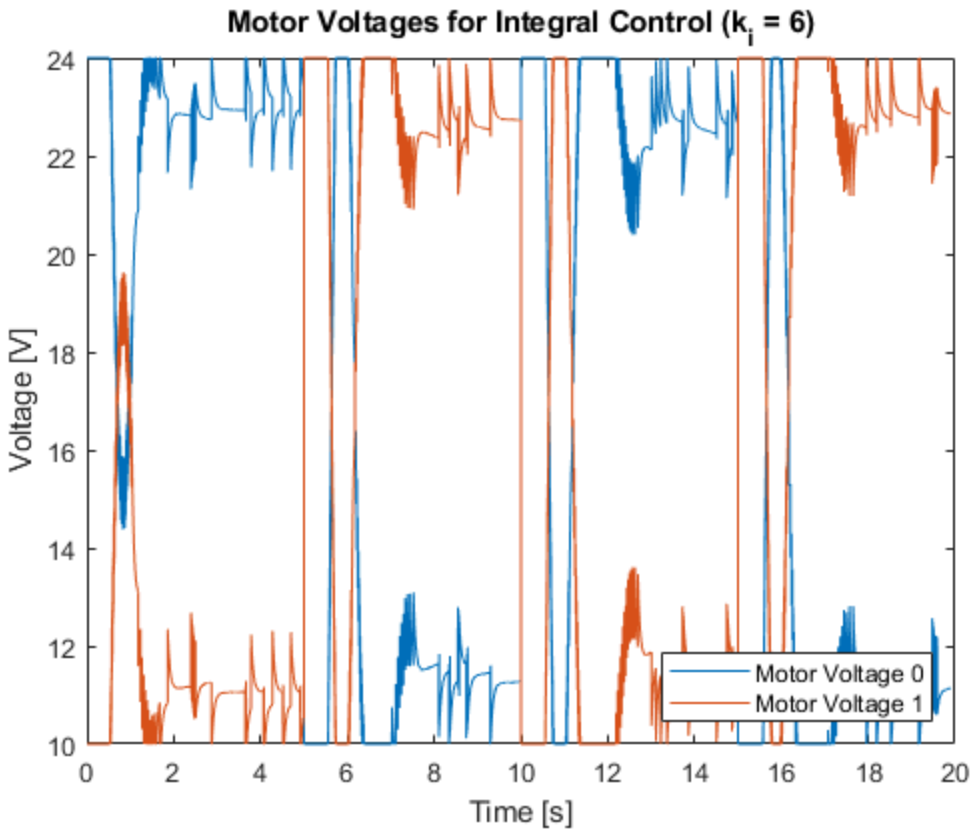
2.4 Integral Gain: Voltage ki=4.5

```
clear
clc
close all

figure(16)
load("11_sq_wave_gain_i_45.mat")
plot(Voltage(:,1), Voltage(:,2))
hold on
plot(Voltage(:,1), Voltage(:,3))
hold on

xlabel("Time [s]")
ylabel("Voltage [V]")
title("Motor Voltages for Integral Control (k_i = 4.5)")
legend("Motor Voltage 0", "Motor Voltage 1", "Location", "SouthEast")
```



2.4 Integral Gain: Voltage ki=6

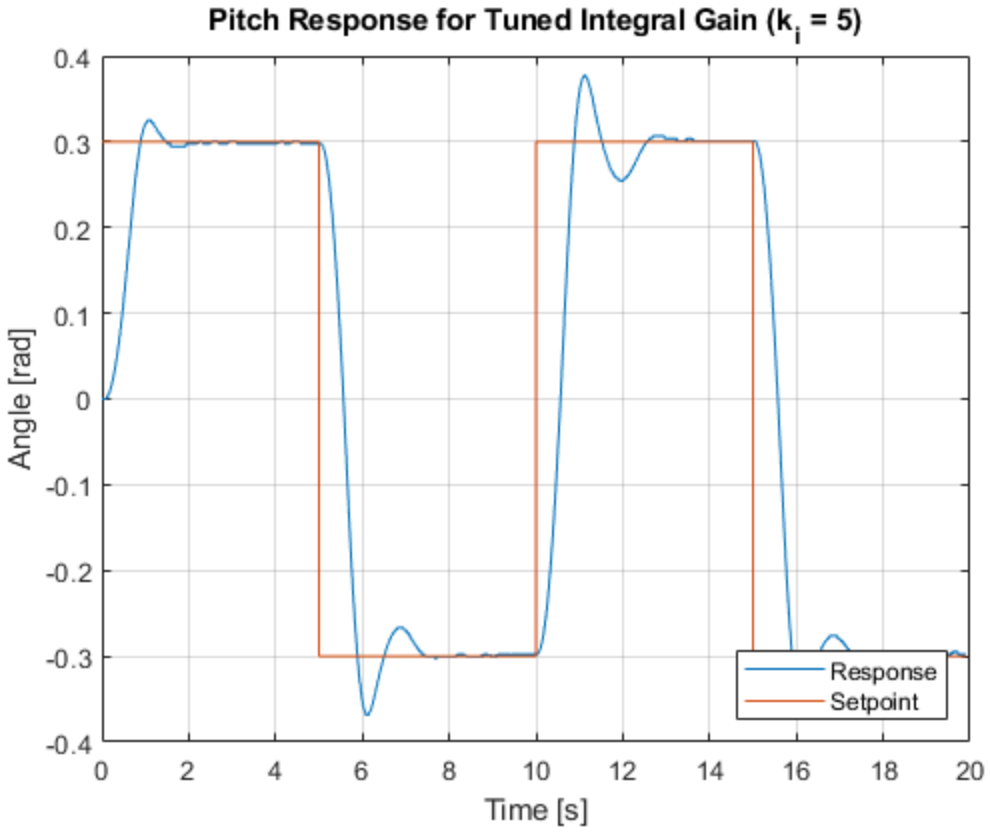
```
clear
clc
close all

figure(17)
load("12_sq_wave_gain_i_6.mat")
plot(Voltage(:,1), Voltage(:,2))
hold on
plot(Voltage(:,1), Voltage(:,3))
hold on

xlabel("Time [s]")
ylabel("Voltage [V]")
title("Motor Voltages for Integral Control (k_i = 6)")
legend("Motor Voltage 0", "Motor Voltage 1", "Location", "SouthEast")
```

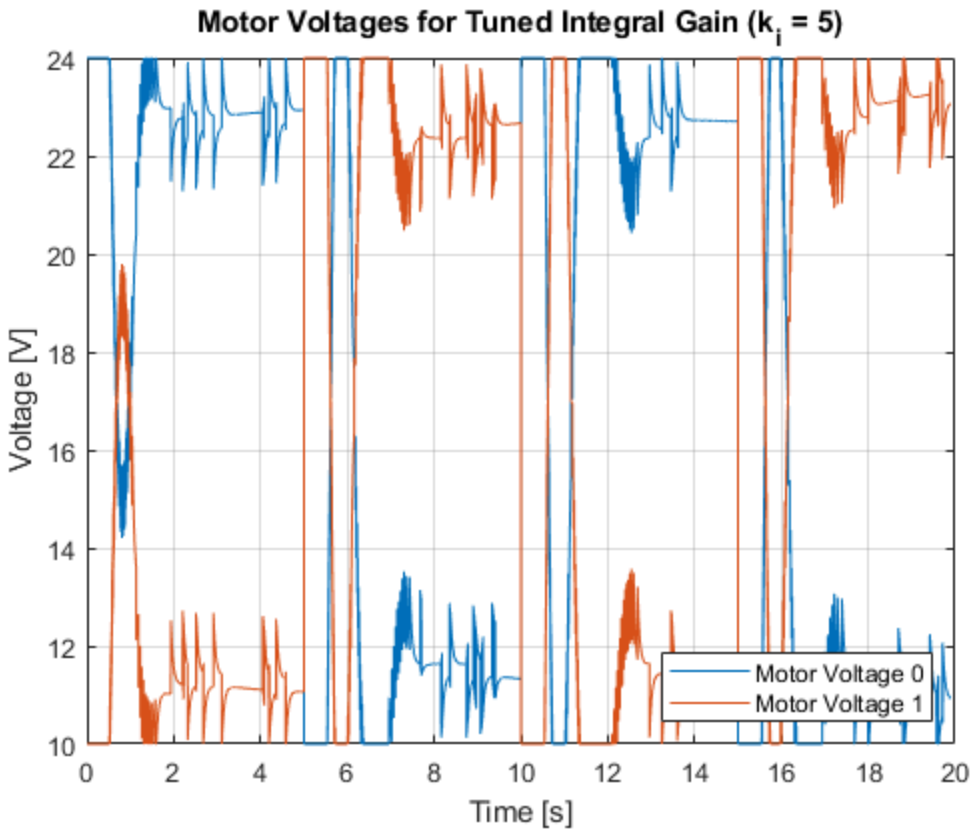


2.4 Integral Control, Step 3, ki=5

```
clc
clear
close all

figure(18)
load("13_sq_wave_gain_3ss_i_5.mat")
plot(Angle(:,1), Angle(:,2))
hold on
plot(Angle(:,1), Angle(:,3))
grid on
xlabel("Time [s]")
ylabel("Angle [rad]")
title("Pitch Response for Tuned Integral Gain (k_i = 5)")
legend("Response", "Setpoint","Location", "SouthEast")

figure(19)
plot(Voltage(:,1), Voltage(:,2))
hold on
plot(Voltage(:,1), Voltage(:,3))
grid on
xlabel("Time [s]")
ylabel("Voltage [V]")
title("Motor Voltages for Tuned Integral Gain (k_i = 5)")
```

```
legend("Motor Voltage 0", "Motor Voltage 1","Location", "SouthEast")
```

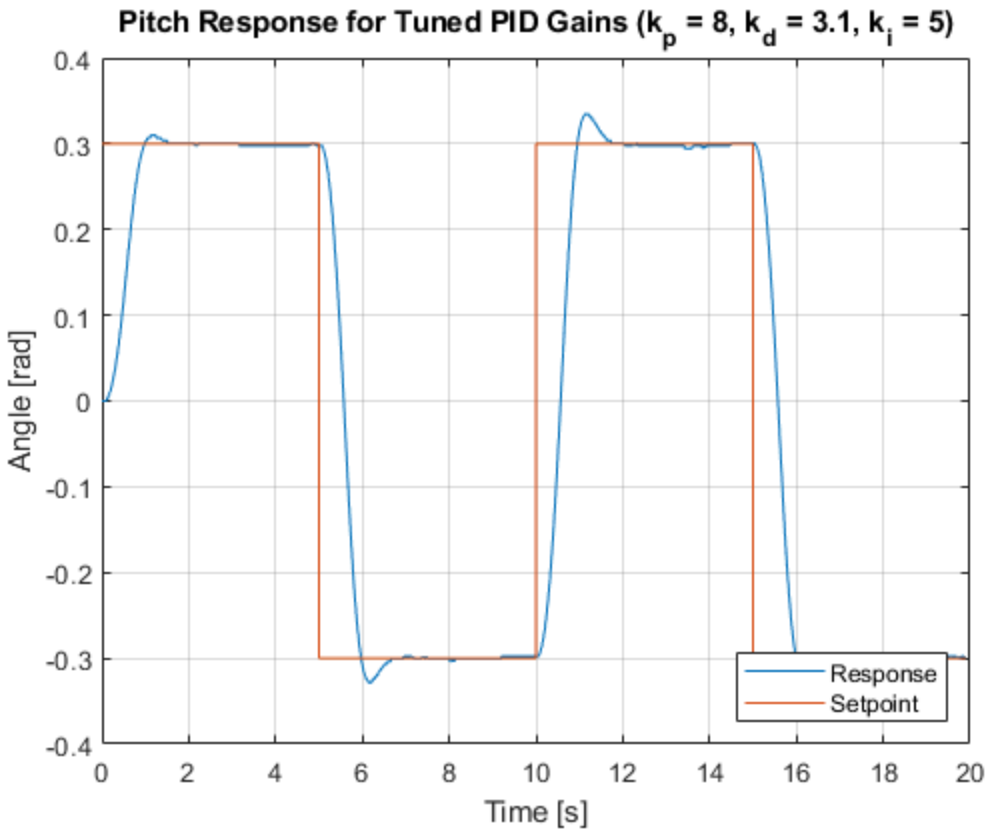


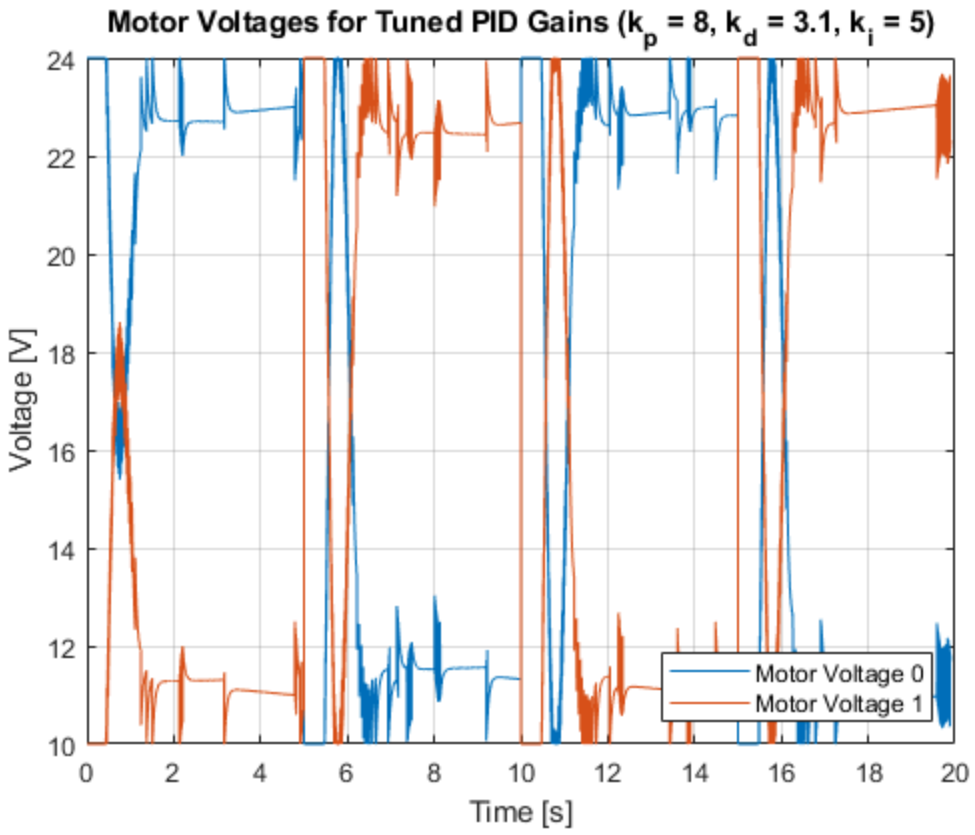


2.5 Response Tuning

```
clc
clear
close all

figure(20)
load("14_sq_wave_gain_k8_d31_i5.mat")
plot(Angle(:,1), Angle(:,2)) % setpoint
hold on
plot(Angle(:,1), Angle(:,3)) % simulated angular speed
grid on
xlabel("Time [s]")
ylabel("Angle [rad]")
title("Pitch Response for Tuned PID Gains (k_p = 8, k_d = 3.1, k_i = 5)")
legend("Response", "Setpoint","Location", "SouthEast")

figure(21)
plot(Voltage(:,1), Voltage(:,2)) % setpoint
hold on
plot(Voltage(:,1), Voltage(:,3)) % simulated angular speed
grid on
xlabel("Time [s]")
ylabel("Voltage [V]")
title("Motor Voltages for Tuned PID Gains (k_p = 8, k_d = 3.1, k_i = 5)")
```

```
legend("Motor Voltage 0", "Motor Voltage 1","Location", "SouthEast")
```

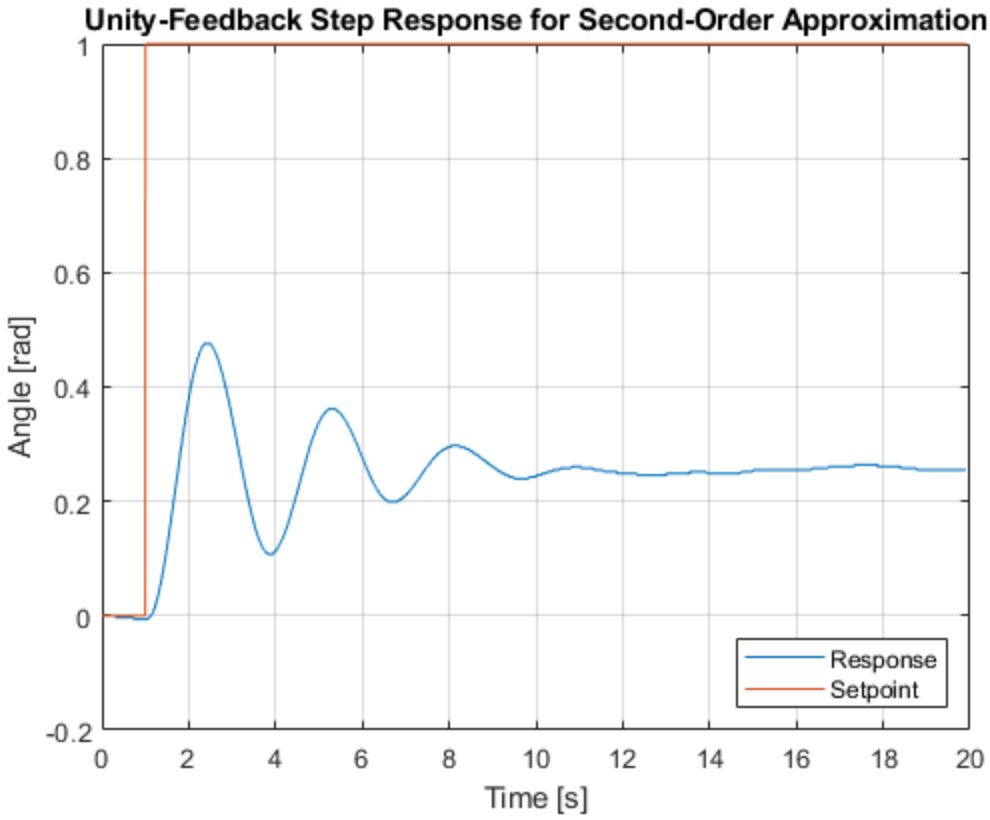


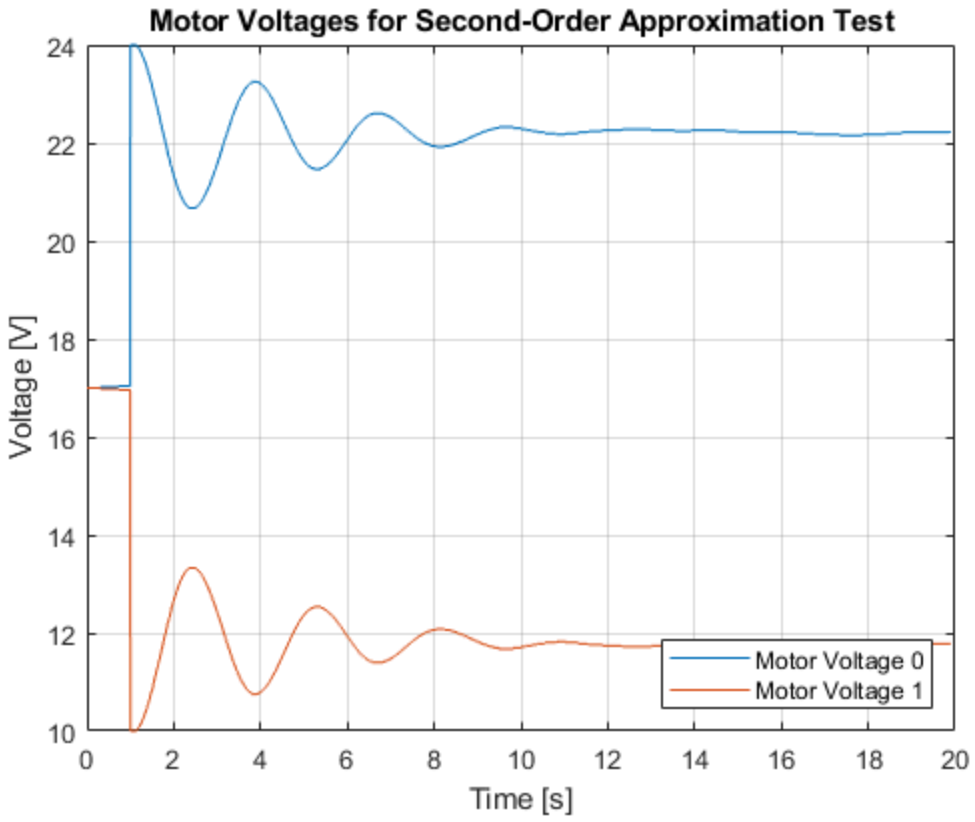


4.1 Second-order appox, Step 3

```
clc
clear
close all

figure(20)
load("15_step_k1_d0_i0.mat")
plot(Angle(:,1), Angle(:,2))
hold on
plot(Angle(:,1), Angle(:,3))
grid on
xlabel("Time [s]")
ylabel("Angle [rad]")
title("Unity-Feedback Step Response for Second-Order Approximation")
legend("Response", "Setpoint","Location", "SouthEast")

figure(21)
plot(Voltage(:,1), Voltage(:,2)) % setpoint
hold on
plot(Voltage(:,1), Voltage(:,3)) % simulated angular speed
grid on
xlabel("Time [s]")
ylabel("Voltage [V]")
title("Motor Voltages for Second-Order Approximation Test")
```

```
legend("Motor Voltage 0", "Motor Voltage 1","Location", "SouthEast")
```

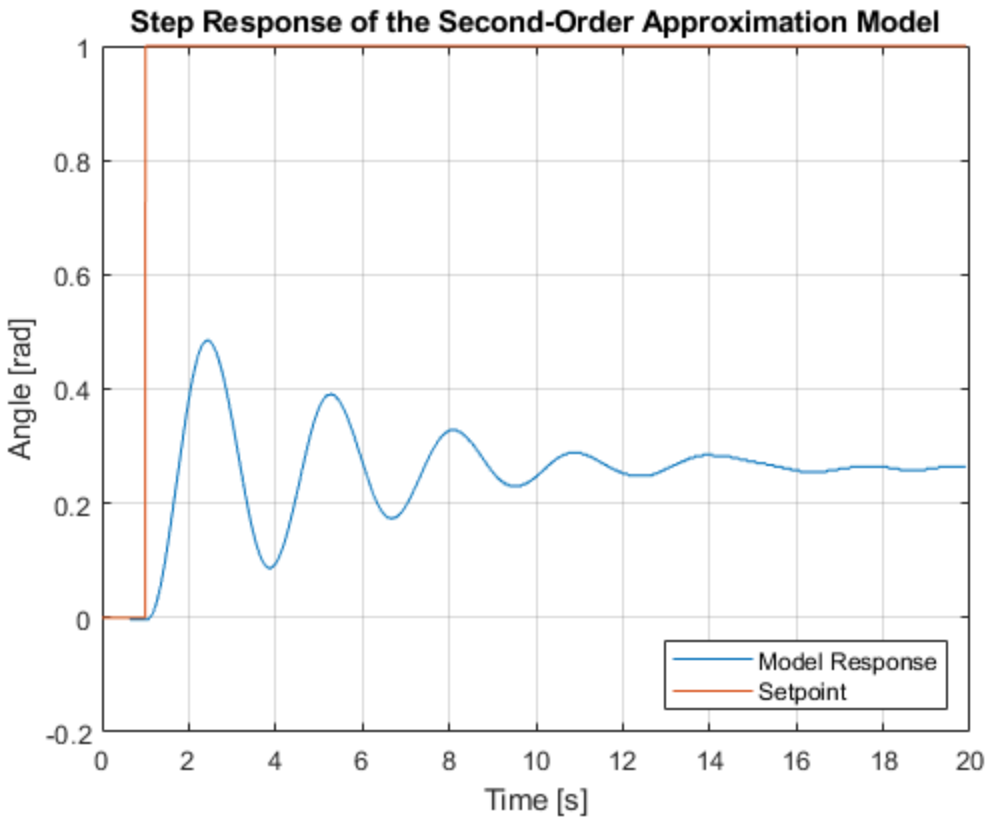


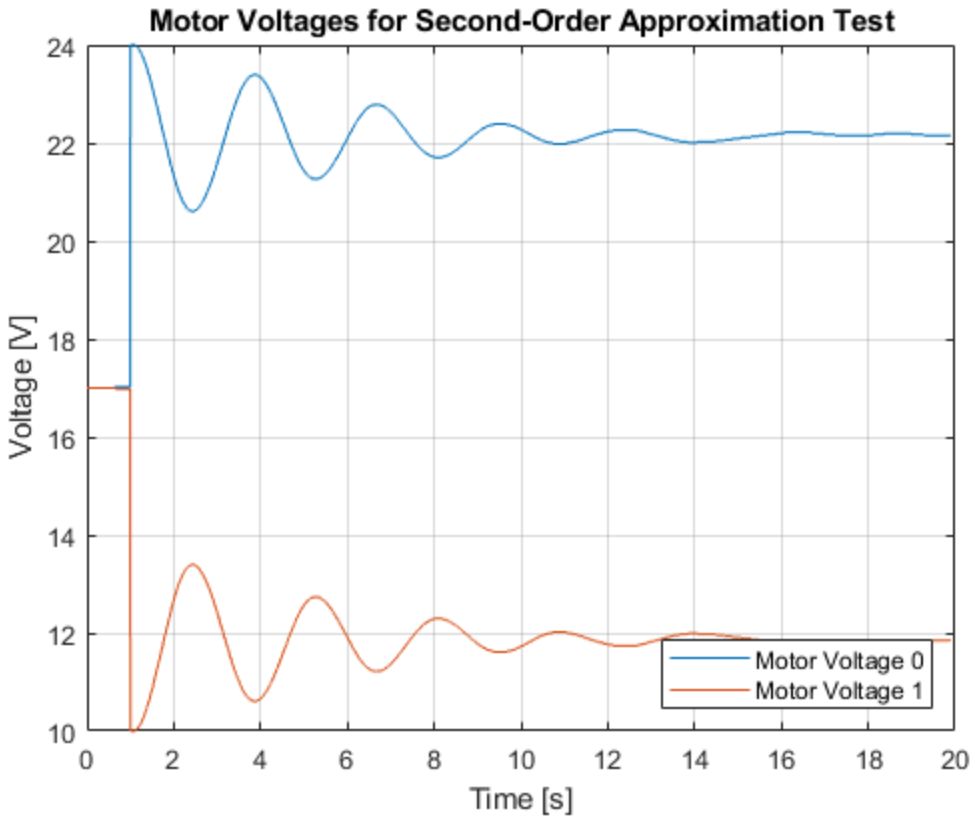


Custom Transfer Function

```
clc
clear
close all

figure(22)
load("16_cstm_tf.mat")
plot(Angle(:,1), Angle(:,2))
hold on
plot(Angle(:,1), Angle(:,3))
grid on
xlabel("Time [s]")
ylabel("Angle [rad]")
title("Step Response of the Second-Order Approximation Model")
legend("Model Response", "Setpoint","Location", "SouthEast")

figure(23)
plot(Voltage(:,1), Voltage(:,2)) % setpoint
hold on
plot(Voltage(:,1), Voltage(:,3)) % simulated angular speed
grid on
xlabel("Time [s]")
ylabel("Voltage [V]")
title("Motor Voltages for Second-Order Approximation Test")
```

```
legend("Motor Voltage 0", "Motor Voltage 1","Location", "SouthEast")
```

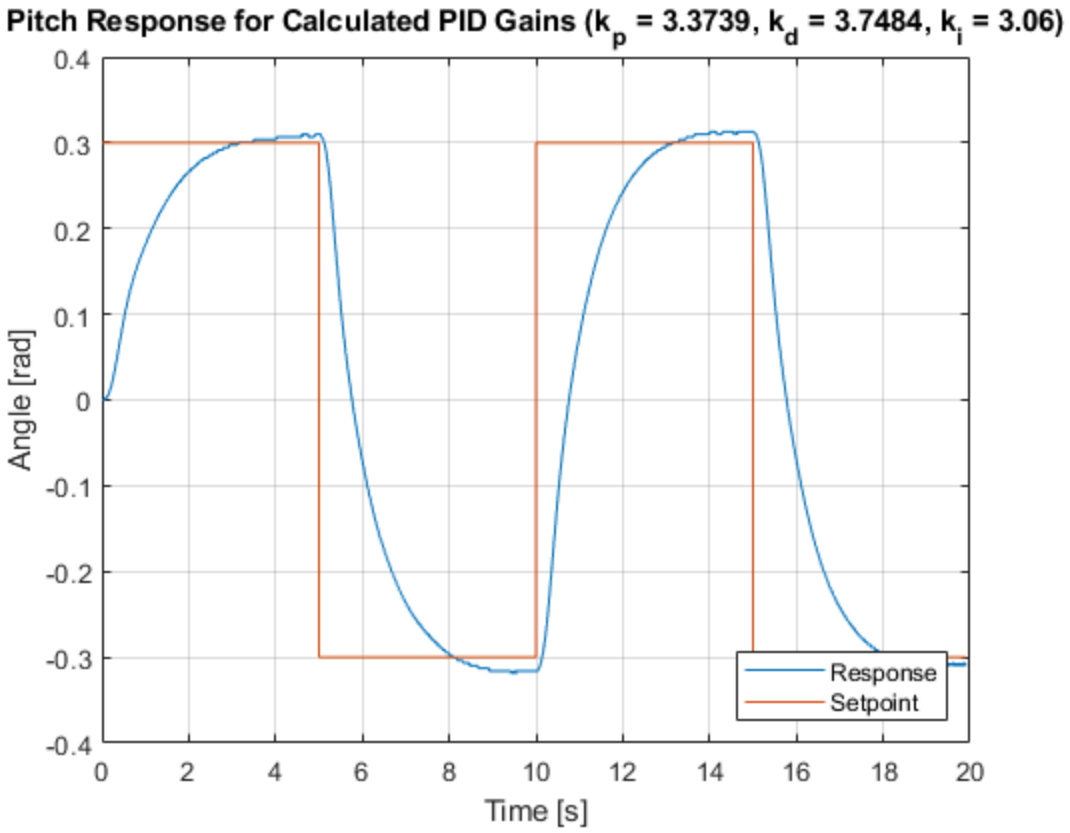


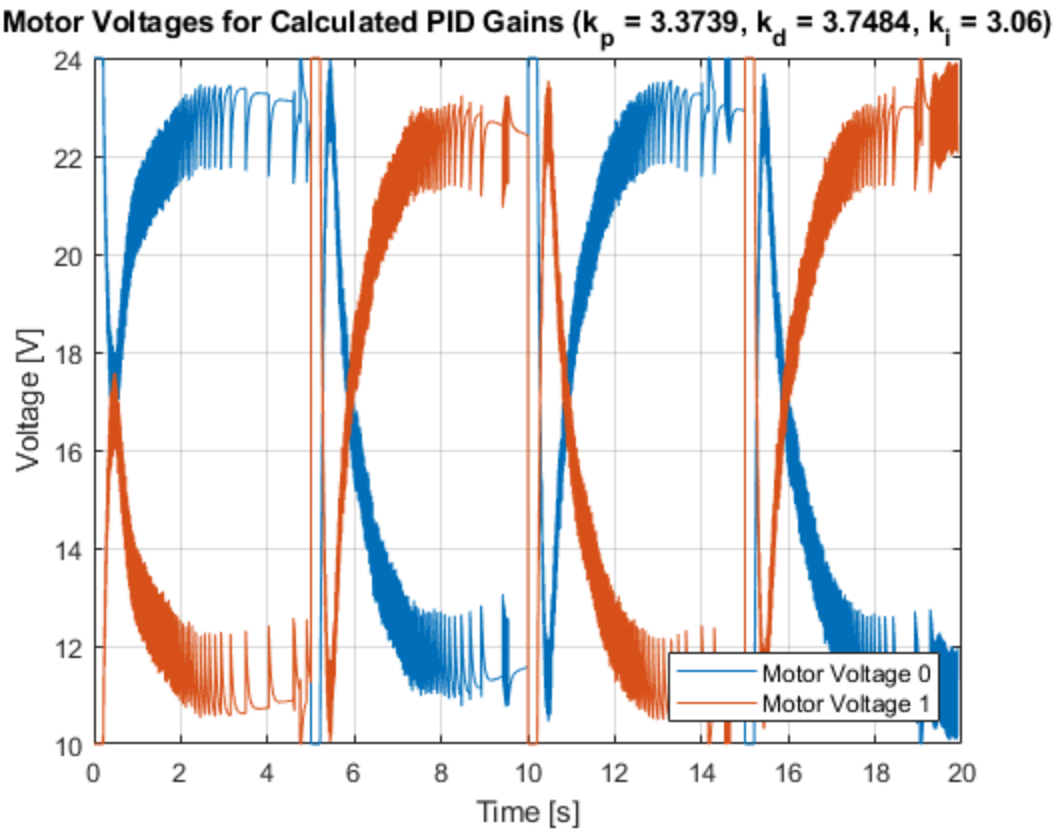


Tuned sq wave

```
clc
clear
close all

figure(22)
load("17_tuned_sq_wave.mat")
plot(Angle(:,1), Angle(:,2))
hold on
plot(Angle(:,1), Angle(:,3))
grid on
xlabel("Time [s]")
ylabel("Angle [rad]")
title("Pitch Response for Calculated PID Gains (k_p = 3.3739, k_d = 3.7484,
 k_i = 3.06)")
legend("Response", "Setpoint","Location", "SouthEast")

figure(23)
plot(Voltage(:,1), Voltage(:,2))
hold on
plot(Voltage(:,1), Voltage(:,3))
grid on
xlabel("Time [s]")
ylabel("Voltage [V]")
```

```
title("Motor Voltages for Calculated PID Gains (k_p = 3.3739, k_d = 3.7484,
 k_i = 3.06)")
legend("Motor Voltage 0", "Motor Voltage 1","Location", "SouthEast")
```





Published with MATLAB® R2022a